OPEN TRAIL

A MINI PROJECT REPORT

Submitted by

ADITHYAN PR [SCM25MCA-2002]

to

The APJ Abdul Kalam Technological University

In partial fulfillment of the requirements for the award of the Degree

of

Master of Computer Application



Department of Computer Science and Engineering

SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY

(*Affiliated to APJ Abdul Kalam Technological University*)

VIDYA NAGAR, PALISSERY, KARUKUTTY

ERNAKULAM - 683576

OCTOBER 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING SCMS SCHOOL OF ENGINEERING AND TECHNOLOGY

(*Affiliated to APJ Abdul Kalam Technological University*)

VIDYA NAGARM PALISSERY, KARUKUTTY

ERNAKULAM - 683582

CERTIFICATE

This is to certify that the report entitled **'OPEN TRAIL'** submitted by **ADITHYAN PR** to the APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of Master of Computer Application is a bonafide record of the project work carried out by him/her under my guidance and supervision.

Project Guide	Project Coordinator	Head of the Department
Dr. Varun G Menon	Mr. Subin PS	Dr. Manish TI

ACKNOWLEDGEMENT

I am greatly indebted to **Dr. Jenson Joseph E**, Principal, SSET, Ernakulam and **Dr. Manish T I**, Head of department, Department of Computer Science and Engineering, SSET, who whole heartedly granted me the permission to do the project

I would like to thank my guide, **Dr. Varun G Menon** and project coordinator, **Mr. Subin P S**, Department of Computer Science and Engineering, SSET who has given me valuable guidance and support all the way.

I would like to express my sincere gratitude to all the teachers of Computer Science Department who gave me moral and technical support. I would like to thank the supporting staff in the Computer lab whose dedicated work kept the lab working smoothly, thus enabling me to have access to various resources which helped me understand more about the project topic. I would also like to thank friends and family members for providing me with necessary resources and support. Last but not the least, I would like to thank God Almighty for helping me to do my project hassle free.

ABSTRACT

Group rides are a popular activity among motorcycle enthusiasts and cycling communities. However, coordinating multiple riders throughout a journey can be challenging due to differences in riding speeds, unexpected stops, and route changes. These situations may cause participants to become separated from the group, making it difficult for the ride leader to monitor everyone and coordinate the journey effectively. OpenTrail is a mobile application developed primarily for motorcycle and cycling groups to simplify group ride planning, coordination, and real-time tracking. The main objective of the project is to provide a common platform through which riders can manage ride sessions, share their live locations, and remain aware of the group's movement throughout a journey.

The application is developed using Flutter and Dart, with Firebase services supporting authentication, data storage, and location sharing. Firebase Authentication provides user sign-in functionality, while Cloud Firestore stores user profiles and ride-related information. Firebase Realtime Database is used to share live location updates among participants during an active ride. GPS technology enables the application to obtain the current location of riders and display their positions on a shared map. The map interface is implemented using Flutter Map with Mapbox tiles, allowing participants to view the geographical positions of other group members. OpenRouteService is integrated for route generation, while a geocoding service supports destination search. The application also integrates the OpenWeather API to provide weather information.

OpenTrail provides features for creating and joining ride sessions, searching for destinations, viewing planned routes, and monitoring the movement of group members. During an active ride, participants can view one another's live locations on the shared map. The application also displays navigation-related information, including remaining distance and estimated travel time, helping riders follow the planned route. The ride leader can coordinate participants and use the SOS feature to indicate an emergency to other members. The interface is designed with the map as its primary component, keeping essential ride information and controls accessible to users.

By integrating ride management, real-time location sharing, route generation, and emergency indication, OpenTrail provides a centralized mobile solution for motorcycle and cycling groups. The project focuses on simplifying group ride coordination and helping participants stay connected and informed throughout their journeys.

TABLE OF CONTENTS

ACKNOWLEDGEMENT **3**

ABSTRACT **4**

TABLES **9**

FIGURES **10**

CHAPTER 1 ..... **11**

INTRODUCTION **11**

1.1 PROJECT OVERVIEW *11*

1.2 OBJECTIVES *12*

1.3 SCOPE OF THE PROJECT *13*

1.4 TECHNOLOGIES USED *14*

CHAPTER 2 ..... **15**

SYSTEM ANALYSIS 15

2.1 PROBLEM DEFINITION *15*

2.2 FEASIBILITY STUDY *16*

2.2.1 TECHNICAL FEASIBILITY 16

2.2.2 OPERATIONAL FEASIBILITY 17

2.3 EXISTING SYSTEM *17*

2.4 PROPOSED SYSTEM *18*

CHAPTER 3 ..... **20**

AGILE DEVELOPMENT METHODOLOGY - [SCRUM] 20

3.1 INTRODUCTION TO AGILE METHODOLOGY *20*

3.2 OVERVIEW OF SCRUM FRAMEWORK *20*

3.3 STAKEHOLDERS AND TARGET USERS *21*

3.4 INITIAL REQUIREMENTS *22*

3.4.1 FUNCTIONAL REQUIREMENTS 22

3.4.2 NON-FUNCTIONAL REQUIREMENTS 22

3.5 USER STORIES *23*

3.6 PRODUCT BACKLOG *24*

3.7 SPRINT PLANNING *26*

3.8 SPRINT-WISE DEVELOPMENT *26*
3.8.1 SPRINT 1 – PROJECT FOUNDATION AND INITIAL INTERFACE 26
3.8.2 SPRINT 2 – NAVIGATION AND SAFETY FEATURES 27
3.8.3 SPRINT 3 – RIDE ANALYTICS AND WAYPOINTS 27
3.8.4 SPRINT 4 – COMMUNITY FEATURES AND RIDE SCHEDULING 28
3.8.5 SPRINT 5 – FINAL IMPROVEMENTS AND VALIDATION 28
3.10 DEFINITION OF DONE *30*
3.11 SUMMARY *31*

CHAPTER 4 ..... **32**

SYSTEM DESIGN 32
4.1 DATA FLOW DIAGRAM *32*
4.1.1 DFD LEVEL 0 33
4.1.2 DFD LEVEL 1 34
4.2 SYSTEM ARCHITECTURE *35*
4.2.1 USERS LAYER 36
4.2.2 USER INTERFACE LAYER 36
4.2.3 APPLICATION LOGIC LAYER 37
4.2.4 BACKEND AND CLOUD SERVICES LAYER 37
4.2.5 EXTERNAL APIS AND LOCATION SERVICES LAYER 37
4.2.6 ARCHITECTURAL COMMUNICATION FLOW 38
4.2.7 ARCHITECTURAL SUMMARY 38
4.3 USE CASE DIAGRAM *39*
4.3.1 ACTORS DESCRIPTION 40
4.3.2 Use Case Descriptions 41

CHAPTER 5 ..... **44**

DATABASE DESIGN 44
5.1 DATABASE SCHEMA *44*

CHAPTER 6 ..... **45**

FRONT-END DESIGN 45
6.1 FRONT-END TECHNOLOGIES *45*
6.1.1 FLUTTER 45
6.1.2 DART 46

6.1.3 FLUTTER MAP AND MAPBOX MAP TILES....46
6.1.4 LIQUID GLASS WIDGETS....47
6.1.5 OPENROUTESERVICE DIRECTIONS API....47
6.1.6 GEOLOCATOR....48
6.1.7 OPENWEATHER API....48
6.1.8 SUPPORTING PACKAGES: HTTP, ENVIRONMENT CONFIGURATION, AND COORDINATES....49
6.1.9 WIKIMEDIA API....50
6.1.10 MAPBOX SEARCH BOX API....50
6.2 FORMS AND INPUT VALIDATION....51
6.2.1 INPUT VALIDATION....52
6.2.2 ERROR HANDLING AND USER FEEDBACK....53
6.3 SCREENSHOTS OF KEY SCREENS....54
***CHAPTER 6*....56**
BACK-END DESIGN....56
7.1 DATABASE MANAGEMENT SYSTEM....56
7.2 SERVER ENVIRONMENT....58
7.3 FRAMEWORKS AND BACK-END SERVICES....59
7.3.1 FIREBASE AUTHENTICATION AND GOOGLE SIGN-IN....59
7.3.2 CLOUD FIRESTORE....59
7.3.3 FIREBASE REALTIME DATABASE....60
7.3.4 DATA FLOW BETWEEN FIREBASE SERVICES....61
7.3.5 SUMMARY OF BACK-END SERVICES....61
***CHAPTER 8*....62**
SYSTEM IMPLEMENTATION....62
8.1 INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)....62
8.2 VERSION CONTROL SYSTEM....63
8.2.1 MAJOR GIT COMMANDS USED IN THE PROJECT....63
8.2.2 GITHUB REPOSITORY DETAILS....64
8.2.3 DEVELOPMENT HISTORY....65
8.2.4 GIT COMMIT HISTORY....66
***CHAPTER 9*....67**

TESTING AND VALIDATION....67
9.1 UNIT TESTING....*67*
9.3 SYSTEM TESTING....*70*
CHAPTER 10**....73**
CONCLUSION....73
10.1 SUMMARY OF WORK....*73*
10.2 FUTURE SCOPE....*74*
CHAPTER 11**....76**
REFERENCES....76
11.1 TECHNICAL DOCUMENTATION....*76*

TABLES

Table 1 - Technologies Used 14
Table 2 - Stakeholders And Target Users 21
Table 3 - User Stories 23
Table 4 - Product Backlog 25
Table 5 - Use Case Description 42
Table 6 - Input Validation 52
Table 7 - Git Commands 64
Table 8 - Unit Testing 68
Table 9 - Integration Testing 70
Table 10 - System Testing 72

FIGURES

Figure 1 - Sprint Review Screen Shots 30
Figure 2 - Github Proof 31
Figure 3 - Dfd Symbols 32
Figure 4 - Dfd Level 0 33
Figure 5 - Dfc Level 1 34
Figure 6 - System Architecture 35
Figure 7 - Use Case Diagram 39
Figure 8 - Database Schema 44
Figure 9 - Open Trail Screen Shots 54
Figure 10 - Open Trail Screen Shots 55
Figure 11 - Git Commit History 66

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

Motorcycle touring and cycling are popular activities among people who enjoy travelling together and exploring new destinations. However, coordinating multiple riders throughout a journey can be challenging due to differences in riding speeds, unexpected stops, and changes in the planned route. These situations may cause participants to become separated from the group, making it difficult for the ride leader to monitor their positions and coordinate the journey effectively. The absence of a shared platform for location tracking and ride management can further complicate group coordination, particularly during long journeys.

GPS technology, digital mapping services, and mobile applications provide practical ways to address these challenges. GPS enables mobile devices to determine their geographical positions, while mapping and routing services help users identify routes to their selected destinations. Combining these technologies with real-time data sharing allows riders to view the locations of other participants and remain informed about the movement of the group during a journey.

OpenTrail is a mobile application developed primarily for motorcycle and cycling groups to simplify group ride planning, coordination, and real-time tracking. The application is developed using Flutter and Dart, with Firebase services supporting user authentication, ride information management, and location sharing. Cloud Firestore stores user profiles and ride-related information, while Firebase Realtime Database supports the sharing of live location updates among participants. The application uses Flutter Map with Mapbox tiles to display riders' positions on a shared map and integrates OpenRouteService for route generation. It also provides destination search, navigation-related information such as remaining distance and estimated travel time, and weather information through the OpenWeather API.

The application allows users to create and join ride sessions, view the live locations of group members, search for destinations, and follow planned routes. It also includes an SOS feature that enables the ride leader to indicate an emergency to other participants. By combining these implemented features into a single mobile platform, OpenTrail aims to simplify group ride management, improve awareness of participants' locations, and help motorcycle and cycling groups coordinate their journeys more effectively.

1.2 OBJECTIVES

The primary objective of OpenTrail is to develop a cross-platform mobile application that simplifies group ride coordination for motorcycle and cycling communities. It aims to provide a common platform for managing ride sessions, scheduling community rides, sharing real-time locations, and accessing route information to help participants stay connected throughout their journeys. The application is designed to support both individual ride coordination and community-based ride planning, allowing users to organize upcoming journeys and connect with other interested riders. By integrating these functionalities into a single application, OpenTrail aims to make group ride organization more convenient and improve coordination among participants and ride leaders.

The main objectives of the project are:

1. **Cross-Platform Application Development:** To develop the application using Flutter and Dart, supporting both Android and iOS platforms.
2. **User-Friendly Interface:** To design a simple and intuitive interface that allows riders to access ride details, map information, and essential application features easily.
3. **Real-Time Location Tracking:** To implement GPS-based location tracking that enables participants to share their current positions and view other group members on a common map.
4. **Mapping and Route Generation:** To integrate Flutter Map, Mapbox tiles, and OpenRouteService to support destination search, route generation, and visualization of planned routes.
5. **Ride Session Management:** To provide functionality for creating and joining active ride sessions, allowing participants to coordinate and take part in organized group journeys.
6. **Community Ride Scheduling:** To provide a community ride feature that allows users to schedule upcoming rides by specifying ride details and the planned date and time. Other interested riders can view these scheduled rides, join them, and participate at the specified time.
7. **Navigation Information:** To display route-related details, including remaining distance and estimated travel time, helping riders monitor their progress along the planned route.
8. **Emergency Indication:** To implement an SOS feature that allows the ride leader to indicate emergency situations to other participants.
9. **Weather Information:** To integrate the OpenWeather API to provide weather information within the application.

1.3 SCOPE OF THE PROJECT

The scope of OpenTrail is to develop a cross-platform mobile application that supports the planning, coordination, and management of group motorcycle rides and cycling journeys. The application provides a shared platform where riders can organize journeys, monitor one another's locations, follow planned routes, and remain informed about the movement of the group. It is primarily intended for motorcycle and cycling communities that require a convenient way to coordinate participants during both planned and ongoing rides.

OpenTrail allows users to create and join active ride sessions, enabling multiple participants to take part in a common journey. The application also includes a Community Ride feature through which users can schedule upcoming rides by providing details such as the destination, date, and time. Other interested riders can view the scheduled rides, join them, and participate at the specified time. This feature supports community-based ride planning and helps riders connect with others who are interested in joining the same journey.

Through GPS-based location tracking, participants can view the positions of other group members on a shared interactive map. The application uses Flutter Map with Mapbox tiles for map visualization and OpenRouteService for route generation. Destination search is also provided to help users select their intended locations. During navigation, the application displays information such as remaining distance and estimated travel time, allowing riders to monitor their progress towards the destination.

The application also incorporates a map tile caching mechanism that stores previously loaded map tiles and route-related data locally on the device. This allows some previously accessed map content to remain visible when internet connectivity is limited or temporarily unavailable. However, the availability of cached map tiles depends on the previously loaded map area, and complete offline map access is not guaranteed.

The application provides ride management functionality for coordinating participants during active sessions. An SOS feature allows the ride leader to indicate emergency situations to other participants. Firebase Authentication is used for user sign-in, while Cloud Firestore manages user profiles, ride details, and community ride information. Firebase Realtime Database supports the sharing of live location updates among participants. The application also integrates the OpenWeather API to provide weather information.

The current scope of OpenTrail includes group ride planning, community ride scheduling, real-time location sharing, route generation, navigation information, limited offline map caching, and essential ride management through a mobile application. Its operation depends on GPS availability, internet connectivity, and the proper functioning of the integrated Firebase and mapping services. The application is intended to improve awareness and coordination among riders but does not replace responsible riding practices or dedicated emergency services. Future enhancements may include more comprehensive offline map support, advanced ride analytics, improved emergency notifications, and additional communication features.

1.4 TECHNOLOGIES USED

OpenTrail is developed using a combination of mobile application development frameworks, cloud-based services, mapping technologies, and external APIs. Each technology is selected to support specific functionalities of the application, including user authentication, ride management, real-time location tracking, route generation, and destination search. Flutter and Dart are used to develop the mobile application, while Firebase provides backend services for data storage and location sharing. Mapping and routing technologies enable riders to view locations and follow planned routes. Additional APIs are integrated to provide weather information and location-related details. The following table presents the technologies used in the development of OpenTrail and their respective purposes.

Technology	Description
Flutter	Framework for application development
Dart	Programming language used with flutter framework
Firebase	Backend services for the application
Flutter Map	Flutter OpenRouteService map package
Mapbox	For displaying map tiles
OpenRouteService API	Routing services
Geolocator	Flutter Package for device GPS location
VS Code	IDE
Git and GitHub	Version control
Open Weather API	For weather information
Mapbox Search box API	For location or place names
Wiki Media API	For displaying location or places images

Table 1 - technologies used

CHAPTER 2

SYSTEM ANALYSIS

2.1 PROBLEM DEFINITION

Group motorcycle rides and cycling trips are popular among enthusiasts, but organizing and managing multiple riders throughout a journey can be challenging. Participants may travel at different speeds, make unexpected stops, or become separated from the group due to route changes. In such situations, the ride leader may find it difficult to monitor the locations of all participants and ensure that everyone remains aware of the group's movement. Coordinating upcoming rides can also be inconvenient when there is no common platform through which interested riders can view scheduled journeys and participate.

Most navigation applications primarily provide directions and route information for individual users. However, group ride coordination requires additional functionality, such as sharing live locations, managing participants, and organizing ride sessions. Relying on separate phone calls or messaging applications to communicate ride details and monitor participants can make the coordination process less convenient. These difficulties can become more significant when groups travel through unfamiliar areas or when a participant requires assistance during a journey.

To address these challenges, OpenTrail is developed as a mobile application specifically for motorcycle and cycling communities. It provides functionality for creating and joining active ride sessions, scheduling upcoming rides through the Community Ride feature, and allowing interested participants to join planned journeys. During active rides, users can view the real-time locations of other participants on a shared map and access route information, including remaining distance and estimated travel time. The application also incorporates route generation, limited map tile caching, weather information, and an SOS feature through which the ride leader can indicate emergency situations to other participants.

Therefore, the problem addressed by OpenTrail is the lack of a centralized platform that combines community ride scheduling, active ride coordination, real-time location sharing, and route-related information. By integrating these implemented functionalities into a single mobile application, OpenTrail aims to simplify group ride organization and improve coordination and awareness among riders throughout their journeys.

2.2 FEASIBILITY STUDY

A feasibility study helps determine whether a proposed system can be developed using the available technologies, resources, and development environment, and whether it can effectively fulfil its intended purpose. It evaluates the practicality of the project by considering its technical requirements, development resources, and usability. In the case of OpenTrail, the feasibility study focuses on evaluating the technologies and services required to develop the mobile application and their suitability for supporting group ride planning, community ride scheduling, real-time location tracking, and navigation.

The study also considers the availability of GPS functionality, internet connectivity, Firebase services, mapping technologies, and external APIs required for the application. These components are assessed to determine whether they can work together to support the implemented features of OpenTrail and provide a practical solution for motorcycle and cycling communities.

2.2.1 TECHNICAL FEASIBILITY

OpenTrail is developed using Flutter and Dart, which provide the necessary tools for building a cross-platform mobile application. Flutter is used to develop the user interface, while Dart handles the application logic and interactions between different components. This combination supports the development of an application for both Android and iOS platforms using a common codebase.

Firebase services are integrated to provide backend functionality. Firebase Authentication handles user authentication, including Google Sign-In, while Cloud Firestore stores user profiles, ride details, and community ride information. Firebase Realtime Database is used to manage and share live location updates among participants during active ride sessions. These cloud-based services provide the required infrastructure without the need to develop a separate backend server.

The application uses Flutter Map with Mapbox tiles to display interactive maps and rider locations. OpenRouteService is integrated for route generation, while Geolocator obtains the device's current geographical position for GPS-based tracking. Geocode Earth supports destination search and autocomplete. The OpenWeather API provides weather information, and a map tile caching mechanism stores previously loaded map content for limited offline access.

The technologies and services used in OpenTrail are established development tools and provide the major components required for its implementation. Their integration makes the proposed system technically feasible. However, the application's functionality depends on factors such as internet

connectivity, GPS availability, device permissions, and the availability of external APIs. The extent of offline map access is also limited by the map tiles previously loaded and cached on the device.

2.2.2 OPERATIONAL FEASIBILITY

OpenTrail is designed for motorcycle riders, cycling groups, and ride leaders who require a convenient platform to organize and coordinate their journeys. The application provides features for creating and joining active ride sessions, scheduling upcoming rides through the Community Ride feature, and allowing interested riders to join planned journeys. These functionalities help users organize group activities and participate in rides through a common platform.

The map-based interface allows participants to view the real-time locations of other riders and access route information during an active journey. Navigation-related details, such as remaining distance and estimated travel time, help riders monitor their progress towards the destination. The application also provides an SOS feature through which the ride leader can indicate emergency situations to other participants. Weather information and limited offline map caching further support the application's ride-related functionality.

The application uses a user-friendly mobile interface that allows users to access its main features without requiring advanced technical knowledge. Users need to provide the necessary location permissions for GPS tracking and maintain an internet connection for services such as authentication, live location sharing, and online route generation. The availability of previously cached map tiles provides limited access to map content when connectivity is unavailable.

Considering its intended users, implemented functionalities, and operating requirements, OpenTrail is operationally feasible as a mobile application for motorcycle and cycling group coordination.

2.3 EXISTING SYSTEM

Existing navigation and communication applications provide useful features for individual travellers, such as route guidance, location sharing, and messaging. However, coordinating a group motorcycle ride or cycling journey often requires users to depend on multiple applications for different activities. This can make ride planning and participant coordination less convenient, particularly when managing a large group.

The main limitations of the existing systems are:

- **Separate Applications:** Navigation, communication, and ride planning may require different applications, making it difficult to manage all activities from a single platform.

- **Group Coordination:** Monitoring multiple riders throughout a journey can be challenging, especially when participants travel at different speeds or become separated.
- **Location Awareness:** Participants may need to rely on separate location-sharing services to view one another's positions during a ride.
- **Route Sharing:** Planning and sharing a common route among multiple riders may require additional communication and coordination.
- **Community Ride Planning:** Organizing upcoming group rides and informing interested participants may require separate communication platforms.
- **Emergency Indication:** Emergency information may not be directly integrated into the group's ride management interface, making it difficult to communicate an emergency within the riding group.

These limitations indicate the need for a centralized application that combines group ride scheduling, real-time location sharing, route information, and ride coordination.

2.4 PROPOSED SYSTEM

The proposed system brings the implemented functionalities together to provide a centralized platform for organizing group rides and managing different stages of a riding activity. While active ride sessions support real-time coordination among participants, the Community Ride feature allows users to discover scheduled journeys and join upcoming rides. This supports both immediate ride coordination and community-based ride planning within the same application.

The application operates through a combination of GPS functionality, internet connectivity, and cloud-based Firebase services. Location permissions are required to access the device's geographical position and support location sharing. The implemented map caching mechanism also retains previously loaded map tiles and route-related data, providing limited access to cached content when connectivity is unavailable. These operating conditions define the practical scope of the proposed system.

The main features of the proposed system are:

- **User Authentication:** Provides user sign-in functionality through Firebase Authentication, including Google Sign-In.
- **Ride Creation and Joining:** Allows users to create new active ride sessions and join existing rides.

- **Community Ride Scheduling:** Enables users to schedule upcoming rides with destination, date, and time details. Interested riders can view and join these rides before the scheduled time.
- **Real-Time Location Tracking:** Allows participants to share their GPS locations and view other group members on a common interactive map.
- **Route Planning and Navigation:** Supports destination search, route generation, route visualization, and the display of remaining distance and estimated travel time.
- **Ride Management:** Provides functionality for coordinating participants and managing active ride sessions.
- **SOS Mechanism:** Allows the ride leader to indicate emergency situations to other participants.
- **Offline Map Caching:** Stores previously loaded map tiles and route-related data locally, providing limited access to cached map content when internet connectivity is unavailable.
- **Weather Information:** Provides weather information through the integration of the OpenWeather API.

The proposed system integrates these functionalities to support different stages of group riding activities, from scheduling upcoming journeys to coordinating participants during active rides. The Community Ride feature provides an opportunity for riders to discover and participate in scheduled group journeys, while active ride sessions support real-time coordination and location sharing. This combination makes OpenTrail suitable for both organized riding communities and smaller groups of riders.

The operation of the application depends on GPS availability, appropriate device location permissions, and internet connectivity for cloud-based services and live location sharing. The implemented map caching mechanism retains previously loaded map tiles and route-related data, providing limited access to cached content when connectivity is unavailable. However, complete offline map access is not guaranteed.

Overall, OpenTrail provides a unified mobile-based approach to group ride organization by bringing the required planning, tracking, and coordination facilities into one application. The proposed system is intended to make managing group riding activities more convenient and accessible for motorcycle and cycling communities.

CHAPTER 3

AGILE DEVELOPMENT METHODOLOGY - [SCRUM]

3.1 INTRODUCTION TO AGILE METHODOLOGY

Agile is a software development approach that focuses on iterative development, continuous improvement and adapting to changing requirements throughout the development process. Instead of completing the entire application in a single development cycle, the project is divided into smaller activities that can be planned, implemented and evaluated progressively.

Agile methodology encourages continuous evaluation of development progress and allows improvements to be introduced based on feedback and changing project requirements. This approach is particularly useful for software projects where different functionalities need to be developed and integrated over time.

For the development of OpenTrail, an Agile-based approach was adopted to organise the implementation of the application's major functionalities. The development activities were divided into smaller work units, allowing the project to progress through requirements identification, feature implementation, testing and review.

The iterative approach helped organise the development of important functionalities such as ride creation, group location tracking, route navigation, SOS functionality and community ride scheduling.

3.2 OVERVIEW OF SCRUM FRAMEWORK

Scrum is an Agile framework used to organise and manage complex software development activities through iterative and incremental development. It provides a structured approach for planning work, implementing selected features and reviewing the progress of the developed product.

In Scrum, development activities are organised into fixed-length iterations known as Sprints. Each Sprint has a specific goal and produces a usable increment of the product. Work for each Sprint is selected from the Product Backlog based on its priority and the development objectives.

The Scrum framework uses different artefacts and events to support development planning, transparency and continuous improvement. The Product Backlog maintains the list of required features, while the Sprint Backlog represents the work selected for a particular Sprint.

In OpenTrail, Scrum was used to organise the development activities into multiple Sprints. Each Sprint concentrated on a specific set of functionalities, beginning with the project foundation and progressing towards navigation, location tracking, safety features and community-based activities.

The development process also involved reviewing the implemented features, identifying areas requiring improvement and preparing subsequent development activities based on the observed progress.

3.3 STAKEHOLDERS AND TARGET USERS

Stakeholders are individuals or groups who have an interest in the development and functionality of a software system. Identifying stakeholders helps understand the requirements of the application and the expectations of its intended users.

The primary stakeholders considered during the development of OpenTrail are the Ride Leader, Riders or Members, and the Project Guide.

Stakeholder / User	**Role / Need**	**Main Interaction**
Ride Leader	Creates and manages a group ride	Creates rides, sets destinations and monitors members
Rider / Members	Joins a ride and shares live location	Joins sessions, tracks ride progress, views the group and shares location
Project Guide	Evaluates requirements and project progress	Reviews implementation, testing and Sprint outcomes

Table 2 - Stakeholders and Target Users

The Ride Leader and Riders represent the primary target users of OpenTrail. Their requirements form the basis for the application's functional features.

The Project Guide provides academic supervision and evaluates the development progress. The review process helps identify areas that require clarification, modification or further implementation.

Understanding these stakeholders provides a foundation for defining the requirements and organising the development activities.

3.4 INITIAL REQUIREMENTS

Requirements identification is an important stage in software development because it establishes the functionalities and quality expectations of the proposed application. The initial requirements of OpenTrail were identified based on the problems associated with group ride coordination and the needs of the intended users.

The requirements were divided into functional and non-functional requirements.

3.4.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations and services that the application must provide to its users.

The initial functional requirements identified for OpenTrail include:

1. The system shall allow users to authenticate and access their accounts.
2. The Ride Leader shall be able to create a ride, and other members shall be able to join it.
3. The Ride Leader shall be able to search for and select a destination and obtain the corresponding route.
4. Members shall be able to share and view real-time GPS locations on a common map.
5. The system shall display ride and rider information and support navigation and route progress.
6. Members shall be able to create and manage community rides.

These requirements establish the essential functional capabilities of the application and provide the basis for defining user stories and backlog items.

3.4.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality attributes and operational expectations of the application.

The initial non-functional requirements identified for OpenTrail are:

- **Performance:** The application should respond quickly and perform its operations efficiently.
- **Compatibility:** The application should work correctly with the integrated services and supported platforms.
- **Reliability:** The system should function consistently without unexpected failures.

- **Scalability:** The application should support future growth in users and functionality.
- **Usability:** The interface should be easy to understand and operate.
- **Portability:** The application should support operation across multiple platforms.

These requirements provide quality considerations that guide the development and evaluation of the application.

3.5 USER STORIES

User stories are short descriptions of application requirements written from the perspective of the intended user. They explain what a user wants to accomplish and why the functionality is important.

In OpenTrail, user stories were prepared to represent the major requirements of the application. Each story was assigned an identification number and priority. Acceptance criteria were also defined to describe the expected outcome of the corresponding functionality.

ID	User Story	Priority	Acceptance Criteria
US01	As a Ride Leader, I want to create a ride so that other riders can join the same session.	High	Ride is created and a joinable session is generated.
US02	As a Rider, I want to join a ride so that I can participate with the group.	High	A valid ride can be joined and the rider appears in the group.
US03	As a Rider, I want to see the real-time member locations so that I can stay aware of the group.	High	Active riders appear and their locations update on the map.
US04	As a Rider, I want to set a destination and view the route so that the group can follow a common ride.	Medium	Destination is selected and route information is displayed.

Table 3 - User Stories

The user stories helped translate the initial requirements into user-oriented development tasks. The priority assigned to each story indicated its relative importance during development planning.

High-priority stories focused on essential ride creation, participation and location-sharing capabilities. The destination and route-related story was assigned medium priority in the initial user story list.

The acceptance criteria provided a basis for evaluating whether the expected functionality was achieved during implementation and testing.

3.6 PRODUCT BACKLOG

The Product Backlog is an organised list of features, improvements and other development requirements associated with a product. It provides an overview of the work required to develop and improve the application.

For OpenTrail, the Product Backlog was prepared by identifying the major functionalities required for the system. Each backlog item was assigned a unique Product Backlog Item (PBI) identifier, priority and story point estimate. A development status was also maintained to indicate the progress of each item.

Story points were used to represent the estimated relative effort associated with individual backlog items.

PBI	Backlog Item / Feature	Priority	Story Points	Status
PBI-01	Project setup and initial UI	High	3	Done
PBI-02	Map interface and rider visualization	High	5	Done
PBI-03	Route planning and turn-by-turn navigation	High	8	Done
PBI-04	Route polylines and rerouting	High	5	Done

PBI	Backlog Item / Feature	Priority	Story Points	Status
PBI-05	SOS and safety features	High	5	Done
PBI-06	Ride analytics	Medium	5	Done
PBI-07	Waypoints	Medium	5	Done
PBI-08	Weather information	Medium	3	Done
PBI-09	Community ride and scheduling	Medium	5	Done
PBI-10	End Ride Analytics	Medium	5	Done

Table 4 - Product Backlog

The Product Backlog contains ten identified development items covering the major functionalities of OpenTrail. High-priority items mainly address the essential application foundation, map interface, navigation and safety features. Medium-priority items include ride analytics, waypoints, weather information and community ride functionality.

The backlog also provides visibility into the development status of individual features. According to the recorded backlog status, nine items were marked as completed, while End Ride Analytics remained in progress at the time of the review.

The Product Backlog served as the basis for selecting development activities during individual Sprints.

3.7 SPRINT PLANNING

Sprint Planning is the activity through which the work to be performed during a Sprint is identified and organised. It involves establishing a Sprint Goal and selecting relevant items from the Product Backlog.

During OpenTrail development, the project activities were divided into multiple Sprints. Each Sprint was planned around a particular development objective, allowing related functionalities to be implemented progressively.

The Sprint plans identified the following elements:

- **Sprint Duration:** The planned period allocated for the Sprint.
- **Sprint Goal:** The main development objective to be achieved.
- **Selected Product Backlog Items:** The backlog items selected for implementation.
- **Planned Deliverable:** The expected outcome of the Sprint.
- **Definition of Done:** The conditions used to consider a feature complete.

For OpenTrail, the Definition of Done specified that a feature should be implemented, optimised, tested and committed to the Git repository.

This provided a common completion criterion for the planned development activities.

3.8 SPRINT-WISE DEVELOPMENT

The development of OpenTrail was organised into multiple Sprints, with each Sprint focusing on specific application functionalities.

3.8.1 SPRINT 1 – PROJECT FOUNDATION AND INITIAL INTERFACE

Sprint Duration: 12 July 2026 – 18 July 2026

Sprint Goal: To establish the OpenTrail project foundation and develop the initial user and map interface.

Selected Product Backlog Items: PBI-01, PBI-02 and PBI-03.

Planned Deliverable: A basic OpenTrail prototype containing the initial project structure, user interface and map interface.

The first Sprint focused on establishing the foundation of the OpenTrail application. The initial project structure and user interface were developed to provide a base for subsequent functionality.

The map interface was also included in the initial development activities to establish the geographical component of the application. The selected backlog items covered project setup, initial interface development and map-related functionality.

This Sprint provided the initial application structure required for the development of the remaining features.

3.8.2 SPRINT 2 – NAVIGATION AND SAFETY FEATURES

Sprint Duration: 19 July 2026 – 1 August 2026 (date range interpreted from the PPT).

Sprint Goal: To implement navigation and mapping features for route planning, rider visualization, ride monitoring and SOS alerts.

Selected Product Backlog Items: PBI-04 and PBI-05.

Planned Deliverable: A functional navigation and mapping module with route visualization, rerouting, rider markers and SOS alerts.

The second Sprint concentrated on improving the geographical and navigation capabilities of OpenTrail. The development activities focused on displaying route information and supporting the visualisation of riders on the map.

Route polylines and rerouting functionality were included to improve route representation and navigation-related operations. SOS and safety features were also part of the planned development activities to provide an emergency indication mechanism within the group ride environment.

The outcome of this Sprint was intended to improve navigation and provide additional safety-related functionality for participating riders.

3.8.3 SPRINT 3 – RIDE ANALYTICS AND WAYPOINTS

Sprint Duration: 2 August 2026 – 8 August 2026

Sprint Goal: To enhance route planning and SOS functionality and introduce supporting information features such as waypoints.

Selected Product Backlog Items: PBI-06 and PBI-07.

Planned Deliverable: An enhanced OpenTrail application with waypoint-based route planning, SOS functionality, improved performance and weather information.

The third Sprint focused on improving the existing application functionalities and introducing additional ride-related information.

Ride Analytics and Waypoints were the backlog items selected for this Sprint. The planned work also identified improvements to route planning, SOS functionality and application performance.

These activities were intended to improve the overall ride experience by providing additional route-related information and supporting more flexible ride planning.

3.8.4 SPRINT 4 – COMMUNITY FEATURES AND RIDE SCHEDULING

Sprint Duration: 9 August 2026 – 23 August 2026

Sprint Goal: To introduce community-based features and ride scheduling functionality.

Selected Product Backlog Items: PBI-08 and PBI-09.

Planned Deliverable: A community module that provides ride-related information and supports scheduled rides.

The fourth Sprint concentrated on introducing community-oriented functionality into OpenTrail. The selected backlog items included Weather Information and Community Ride and Scheduling.

The planned community module was intended to provide information about organised rides and support the scheduling of future group activities. This extended the application beyond active ride coordination by introducing functionality related to discovering and organising upcoming rides.

The Sprint contributed to the development of community-based features within the application.

3.8.5 SPRINT 5 – FINAL IMPROVEMENTS AND VALIDATION

Sprint Duration: 25 August 2026 - 11 September 2026

Sprint Goal: To complete the navigation, group monitoring and validation features and prepare a stable OpenTrail increment.

Selected Development Activities: Improvement of ride information presentation, optimisation of real-time location tracking, completion of remaining ride-related functionality and comprehensive application testing.

Completed Deliverable: An improved OpenTrail application with real-time group tracking, destination routing, route progress, enhanced ride monitoring and validated user workflows.

The fifth Sprint focused on completing the remaining development activities and improving the overall stability of the OpenTrail application. The primary objective was to refine the existing functionalities, address observations identified during the previous review and prepare a more complete application increment.

One of the major activities involved improving the presentation of group and ride information. The ride information card and group sheet were refined to provide users with a clearer representation of relevant ride details and participating members.

The real-time tracking functionality was also improved to maintain responsive location updates during active ride sessions. Attention was given to location update frequency and database communication to improve the overall tracking experience.

Testing and validation formed another important part of this Sprint. The application was evaluated through black-box testing, User Acceptance Testing, navigation validation and regression checks. The testing activities covered important user workflows, including ride creation, joining an existing ride, sharing live locations and following navigation.

The Sprint also focused on completing the remaining ride-related functionality and integrating the developed modules into a stable application workflow.

The completion of Sprint 5 resulted in an improved OpenTrail application with enhanced group monitoring, navigation and ride management capabilities. It also provided an opportunity to validate the overall interaction between the application's major functionalities.

3.9 Sprint Review

Sprint Review is an opportunity to inspect the developed product increment and evaluate the progress achieved during a Sprint. It provides an opportunity to demonstrate implemented functionalities and collect observations that can guide subsequent development.

For OpenTrail, the Sprint Review presentation included a working demonstration of the application. The demonstrated workflow covered user sign-in, ride creation or joining, viewing ride details and observing rider locations on the map.

The demonstration provided a practical representation of the application's major functionalities and helped evaluate the progress of the implementation.

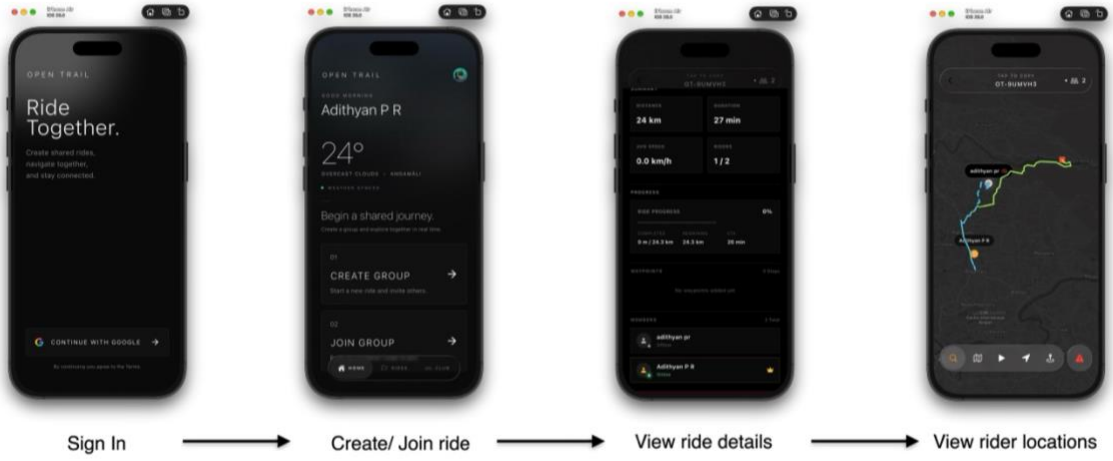


Figure 1 - Sprint Review Screen Shots

The demonstration illustrated the interaction between different application modules. It showed how users could access the application, participate in a ride session, view ride information and observe the geographical positions of other riders.

The working demonstration also provided an opportunity to identify areas requiring further improvement before the project was considered complete.

3.10 DEFINITION OF DONE

The Definition of Done represents the criteria used to determine whether a development item has reached the required level of completion.

For OpenTrail, the Definition of Done specified that a feature should be:

1. Implemented according to the identified requirement.
2. Optimised to support the intended application operation.
3. Tested to verify its expected functionality.
4. Committed to the project's Git repository.

These conditions provided a common basis for evaluating the completion of development tasks during the planned Sprints.

The use of a defined completion criterion helped maintain consistency in the development process and ensured that implementation work was accompanied by testing and version control activities.

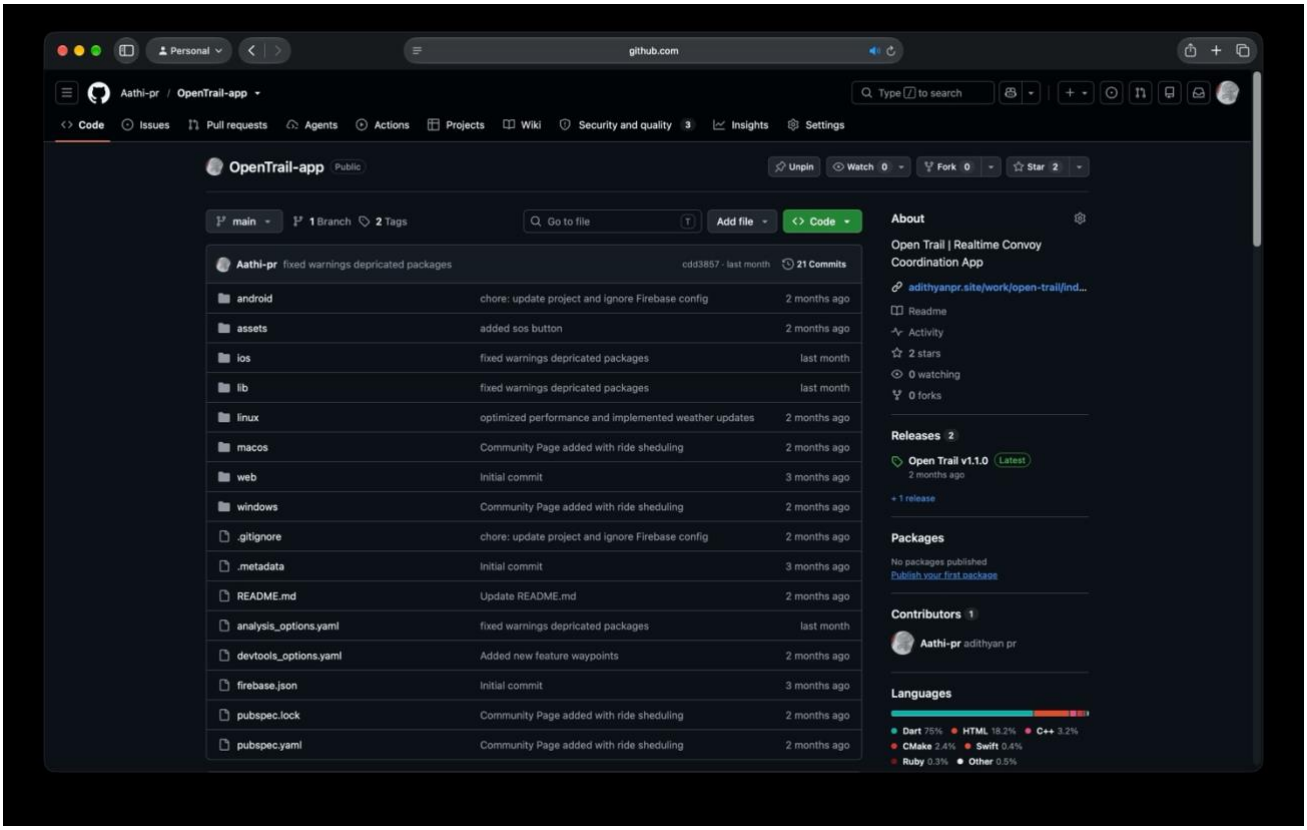


Figure 2 - GitHub Proof

3.11 SUMMARY

The Scrum-based development approach provided a structured method for planning and organising the development of OpenTrail. The initial requirements were converted into user stories and Product Backlog items, which were subsequently used to plan the individual Sprints.

The development activities progressed through project setup, map interface development, route planning, navigation, safety functionality, ride analytics, waypoints and community ride scheduling.

Sprint reviews provided opportunities to demonstrate the working application, evaluate implementation progress and identify areas requiring improvement. The feedback collected during the review was used to prepare additional development activities and validation plans.

Overall, the iterative development approach helped organise the implementation of OpenTrail into manageable stages and provided a systematic way to track development progress and improvements.

CHAPTER 4

SYSTEM DESIGN

4.1 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system. It illustrates the processes that transform data, the external entities that interact with the system, the data stores used to maintain information, and the data flows connecting these components. A DFD helps to understand the internal working of a system by representing its major functions and the movement of information between different components. It is commonly used during system analysis and design to provide a clear overview of the system's data processing activities.

DFDs use standard graphical symbols to represent different components of a system. Each symbol has a specific purpose in describing how information enters, moves through, and is stored within the system. The four basic symbols used in a Data Flow Diagram are:

1. External Entity: Represents a person, organization, or external system that interacts with the application by providing input or receiving output.

2. Process: Represents an activity or operation that transforms incoming data into meaningful output.

3. Data Flow: Represents the movement of information between external entities, processes, and data stores. It is indicated using arrows.

4. Data Store: Represents a location where information is stored for future retrieval and processing

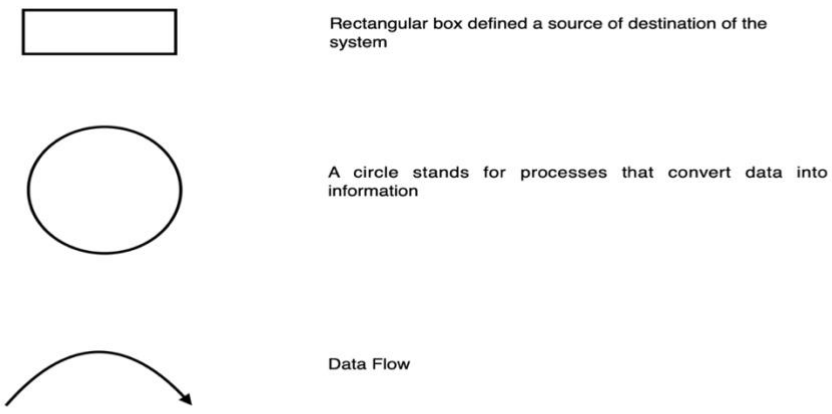


Figure 3 - DFD symbols

4.1.1 DFD LEVEL 0

A Level 0 DFD, also known as a Context Diagram, provides a high-level representation of the entire system as a single process. It illustrates the interaction between the OpenTrail application and its external entities, showing the major inputs and outputs exchanged with the system. This diagram provides an overall understanding of the system boundary and the external interactions without describing its internal processes in detail.

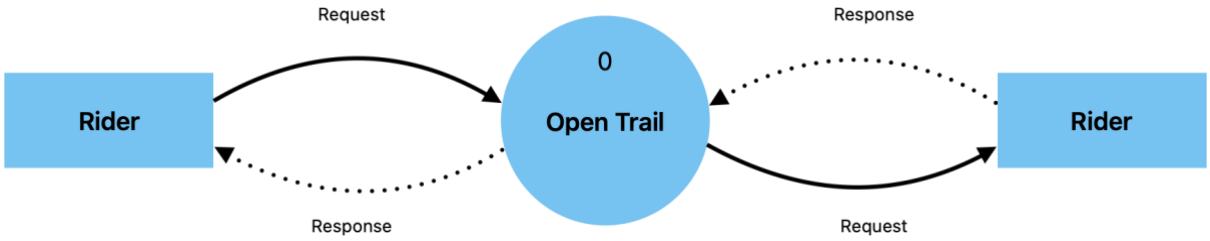


Figure 4 - DFD Level 0

4.1.2 DFD LEVEL 1

A Level 1 DFD provides a more detailed representation by dividing the main system process into smaller sub-processes. It illustrates the major functional activities of the application and the flow of information between these processes, external entities, and data stores. In OpenTrail, the Level 1 DFD represents the main operations involved in user interaction, ride management, community ride activities, and location tracking. It provides a clearer understanding of how different functional components exchange and process information within the application.

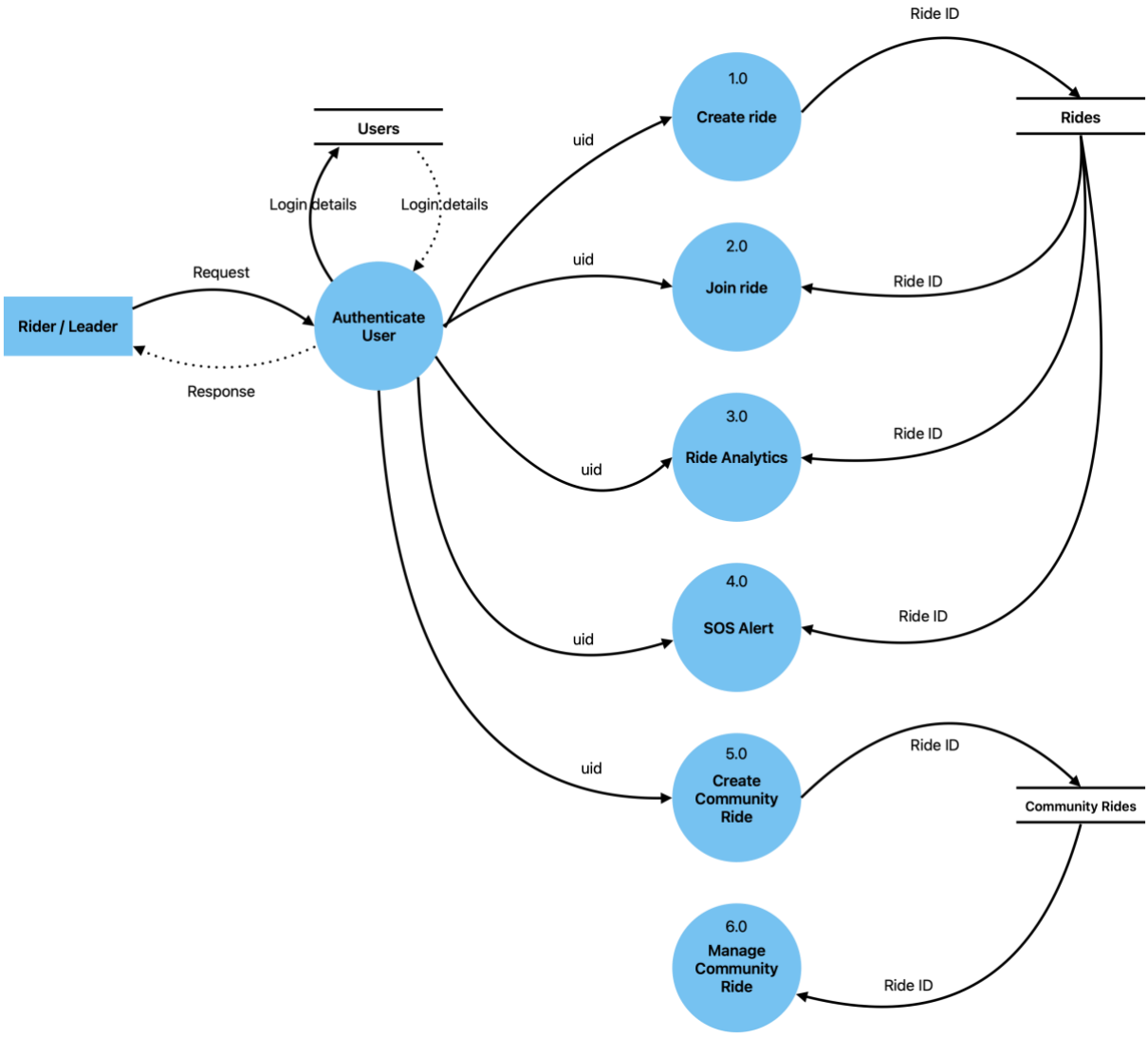


Figure 5 - DFC Level 1

4.2 SYSTEM ARCHITECTURE

System architecture defines the overall structure of a software application and describes how its different components are organised and connected to achieve the intended functionality. It provides a high-level representation of the system by identifying its major layers, responsibilities and interactions.

OpenTrail follows a layered architectural approach in which the application is divided into different functional components. Each layer performs a specific responsibility and communicates with other layers to support the overall operation of the application. This architecture separates user interaction, application processing, backend communication and external service integration.

The architecture is designed to support the core requirements of OpenTrail, including user interaction, ride coordination, real-time location sharing and geographical information management. The separation of responsibilities also improves the maintainability and organisation of the application.

The overall system architecture of OpenTrail illustrates the relationship between users, the mobile application, application logic, backend services and external APIs. It provides a high-level view of how the different components work together to deliver the application's functionalities.

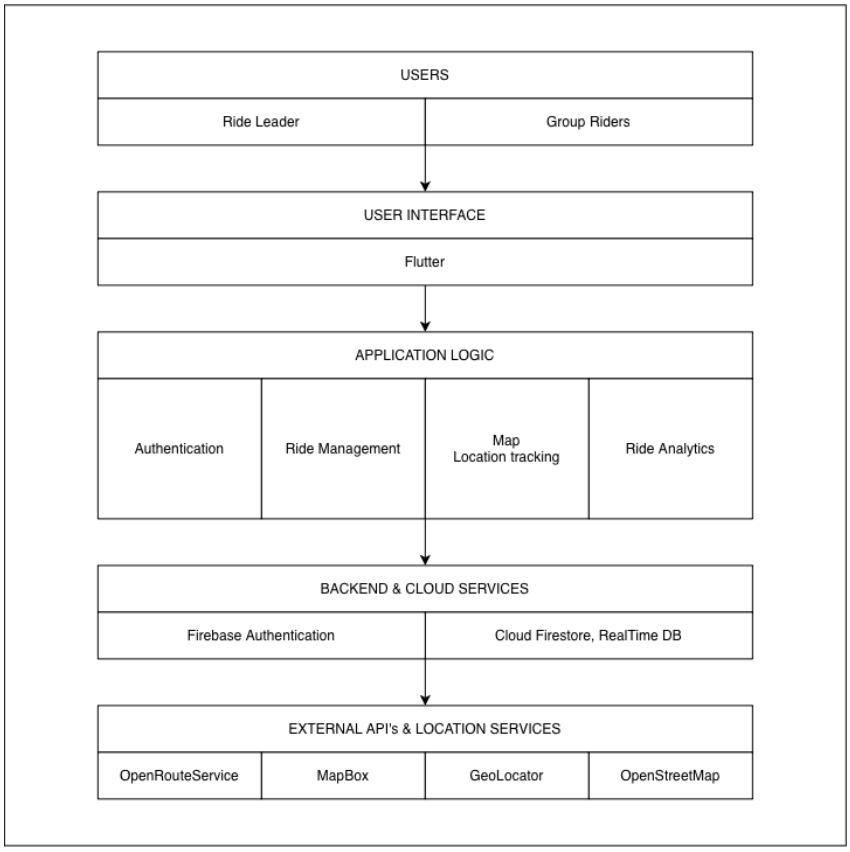


Figure 6 - System Architecture

As illustrated in Figure 4, the architecture consists of five major layers: Users, User Interface, Application Logic, Backend and Cloud Services, and External APIs and Location Services.

The interaction begins at the user layer, where the Ride Leader and Group Riders access the application through the Flutter-based user interface. User actions are processed by the application logic layer, which coordinates the required operations and communicates with the backend and external services.

The backend layer manages application-related data and authentication, while external services provide the supporting capabilities required by the system. The results obtained from these components are processed by the application and presented through the user interface.

This layered organisation provides a structured approach to managing the different responsibilities of OpenTrail.

4.2.1 USERS LAYER

The Users Layer represents the individuals who interact with the OpenTrail application. It consists of two primary user categories: the Ride Leader and Group Riders.

The Ride Leader initiates and coordinates group ride sessions, while Group Riders participate in the sessions by joining an existing ride. Both categories interact with the application through the same mobile interface, with available operations depending on their responsibilities within the ride.

This layer represents the starting point of user interaction and establishes the relationship between the application's functionalities and its intended users.

4.2.2 USER INTERFACE LAYER

The User Interface Layer provides the interaction between users and the application. It is developed using Flutter and acts as the presentation layer of OpenTrail.

This layer is responsible for displaying application screens, collecting user inputs and presenting information received from the application logic. It provides access to the different functionalities of the system through an organised mobile interface.

The user interface communicates with the application logic whenever users perform operations such as accessing ride information, initiating ride-related activities or interacting with the map interface.

By separating presentation responsibilities from application processing, this layer helps maintain a clear distinction between the visual components and the underlying application functionality.

4.2.3 APPLICATION LOGIC LAYER

The Application Logic Layer represents the central functional layer of OpenTrail. It processes user interactions and coordinates the operations required to execute different application features.

As represented in Figure 5.1, this layer consists of four major functional modules.

Authentication: Handles the application's authentication-related operations and coordinates communication with the authentication service.

Ride Management: Manages the operations associated with creating, joining and maintaining ride sessions.

Map Location Tracking: Coordinates geographical operations, including location updates and the presentation of participating riders on the map.

Ride Analytics: Organises and presents relevant ride-related information to users.

These modules divide the application's responsibilities into separate functional areas. This modular organisation makes the application logic easier to understand, maintain and extend.

4.2.4 BACKEND AND CLOUD SERVICES LAYER

The Backend and Cloud Services Layer provides the infrastructure required for authentication and application data management. OpenTrail uses Firebase services to support these operations.

This layer consists of Firebase Authentication, Cloud Firestore and Firebase Realtime Database. Each component supports a specific backend responsibility within the application.

The application logic communicates with these services to perform the required backend operations. Authentication-related requests are handled through the authentication service, while application information and frequently changing data are managed through the respective database services.

The separation of backend responsibilities allows the application to manage persistent information and real-time updates through appropriate cloud services without maintaining a separate custom backend server.

4.2.5 EXTERNAL APIS AND LOCATION SERVICES LAYER

The External APIs and Location Services Layer represents the third-party services integrated into OpenTrail to support geographical and location-related functionality.

As shown in Figure 4, this layer includes OpenRouteService, Mapbox, Geolocator and OpenStreetMap.

These services are accessed by the application logic whenever external geographical information or device location capabilities are required. The information obtained from these services is processed by the application and incorporated into the relevant application features.

This layer extends the capabilities of the mobile application by providing access to external resources without requiring all geographical operations to be implemented internally.

4.2.6 ARCHITECTURAL COMMUNICATION FLOW

The different layers of OpenTrail communicate to perform the operations required by the application. The overall communication process can be described as follows:

1. **User Interaction:** The Ride Leader or Group Rider performs an operation through the mobile application interface.
2. **Request Processing:** The user action is received by the application logic layer, where the corresponding functional module handles the request.
3. **Backend Communication:** When application data or authentication services are required, the application communicates with the appropriate Firebase component.
4. **External Service Communication:** When geographical information, routing or device location capabilities are required, the application interacts with the relevant external service or location component.
5. **Response Processing:** The information received from backend services and external APIs is processed by the application logic.
6. **Information Presentation:** The processed information is presented to users through the Flutter user interface.

This communication structure enables the different architectural layers to work together while maintaining separation between presentation, processing and data management responsibilities.

4.2.7 ARCHITECTURAL SUMMARY

The system architecture of OpenTrail establishes a structured organisation of the application's major components. The layered approach separates user interaction, functional processing, backend communication and external service integration.

The architecture provides a clear relationship between the Ride Leader, Group Riders, Flutter application, Firebase infrastructure and geographical services. Each layer contributes to the overall operation of the system while maintaining its own responsibilities.

This organisation provides a maintainable foundation for the development of OpenTrail and supports the integration of its core group ride coordination functionalities.

4.3 USE CASE DIAGRAM

A Use Case Diagram is a Unified Modelling Language (UML) behavioural diagram that represents the interactions between external actors and the functionalities provided by a system. It illustrates the system boundary, identifies the actors and shows the different operations that can be performed.

The Use Case Diagram of OpenTrail represents the major functionalities available to Riders and Ride Leaders. It illustrates how both actors interact with the application to participate in group rides, manage ride sessions and access navigation-related features.

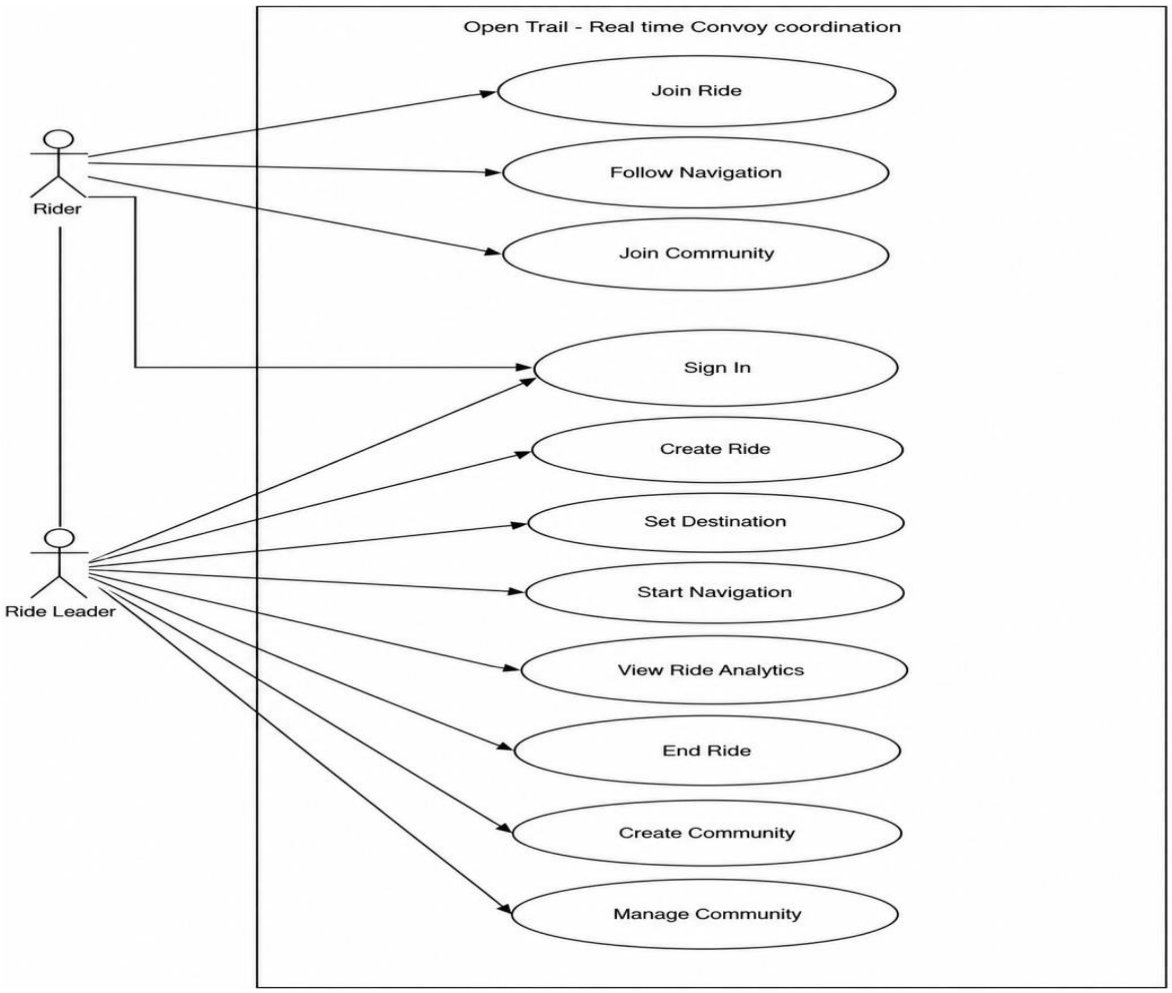


Figure 7 - Use Case Diagram

The diagram identifies two primary actors:

- **Rider:** Represents a participant who uses the application to join existing rides, follow navigation and participate in community activities.
- **Ride Leader:** Represents the user responsible for initiating and managing group ride sessions and accessing ride-related management functionalities.

The diagram also identifies the major use cases associated with each actor, providing an overview of the functional capabilities of OpenTrail.

4.3.1 ACTORS DESCRIPTION

Rider

A Rider is a user who participates in an existing group ride or accesses the available functionalities of the application. Riders can authenticate themselves, join an active ride and follow the navigation information shared during the session.

The Rider can also access community-related functionalities to discover and participate in scheduled group rides. These operations allow participants to coordinate with other riders through the application.

The major use cases associated with the Rider are:

- Sign In
- Join Ride
- Follow Navigation
- Join Community

Ride Leader

The Ride Leader is responsible for initiating and managing group ride sessions. The leader creates a ride and provides the necessary destination information to coordinate the participants.

The Ride Leader can start navigation, monitor ride-related information and terminate an active ride session. The actor also has access to community management functionalities for creating and managing community rides.

The major use cases associated with the Ride Leader are:

- Sign In
- Create Ride
- Set Destination
- Start Navigation
- View Ride Analytics
- End Ride
- Create Community
- Manage Community

4.3.2 Use Case Descriptions

Use Case Descriptions provide additional information about individual functionalities represented in the Use Case Diagram. They describe the actors involved, the conditions required before executing an operation and the expected outcome after its completion.

The following table presents the use cases identified for OpenTrail.

ID	USE CASE	ACTOR	PRECONDITION	POSTCONDITION
UC-01	Sign In	Rider	App + Internet	User authenticated
UC-02	Create Ride	Ride Leader	Authenticated	Ride created + Ride ID generated
UC-03	Join Ride	Rider	Authenticated + Ride ID	Rider joins ride
UC-04	Set Destination	Ride Leader	Ride created	Destination shared
UC-05	Start Navigation	Ride Leader	Active ride + destination	Navigation starts

ID	USE CASE	ACTOR	PRECONDITION	POSTCONDITION
UC-06	Follow Navigation	Rider	Navigation active	Rider follows route
UC-07	Ride Analytics	Rider / Leader	Active ride + Internet	Analytics displayed
UC-08	End Ride	Ride Leader	Active ride	Ride terminated

Table 5 - Use Case Description

Sign In

The Sign In use case allows users to authenticate themselves before accessing the application's functionalities. The operation requires an available application connection and internet access. After successful authentication, the user is recognised as an authenticated application user.

Create Ride

The Create Ride use case allows the Ride Leader to initiate a new group ride session. The operation requires the leader to be authenticated. After successful creation, a new ride session is established and a unique Ride ID is generated for identifying the session.

Join Ride

The Join Ride use case enables a Rider to participate in an existing group ride. The Rider must be authenticated and provide the required Ride ID. Once the operation is completed successfully, the Rider becomes a participant in the selected ride session.

Set Destination

The Set Destination use case allows the Ride Leader to specify the destination associated with an active ride. The ride must already have been created. After the destination is selected and shared, the relevant destination information becomes available for the ride.

Start Navigation: The Start Navigation use case allows the Ride Leader to initiate navigation for an active ride. The operation requires an active ride and an assigned destination. After navigation starts, the route information becomes available to support the navigation process.

Follow Navigation

The Follow Navigation use case allows a Rider to follow the route associated with an active ride. Navigation must already be active before the Rider accesses the route information. The expected outcome is that the Rider can follow the route shared through the application.

Ride Analytics

The Ride Analytics use case allows the Rider or Ride Leader to view available ride-related analytical information. It requires an active ride and an internet connection. After the operation is completed, the relevant analytical information is displayed to the user.

End Ride

The End Ride use case allows the Ride Leader to terminate an active group ride session. The operation requires an existing active ride. Once completed, the ride is marked as terminated, indicating that the session is no longer active.

Create Community

The Create Community use case allows the Ride Leader to create a community-related ride activity. It provides a way to organise scheduled group riding activities and make them available to interested participants.

Manage Community

The Manage Community use case represents the management of community-related activities by the Ride Leader. It supports the organisation and maintenance of community ride information through the application.

Summary

The Use Case Diagram provides a functional overview of OpenTrail by identifying its primary actors and their interactions with the application. The Rider and Ride Leader actors represent the two main user categories, each having access to specific functionalities based on their responsibilities.

The associated Use Case Descriptions provide further clarification regarding the preconditions and expected outcomes of the identified operations. Together, these representations establish a clear understanding of the functional behaviour of OpenTrail and provide a foundation for the subsequent implementation and testing activities.

CHAPTER 5

DATABASE DESIGN

5.1 DATABASE SCHEMA

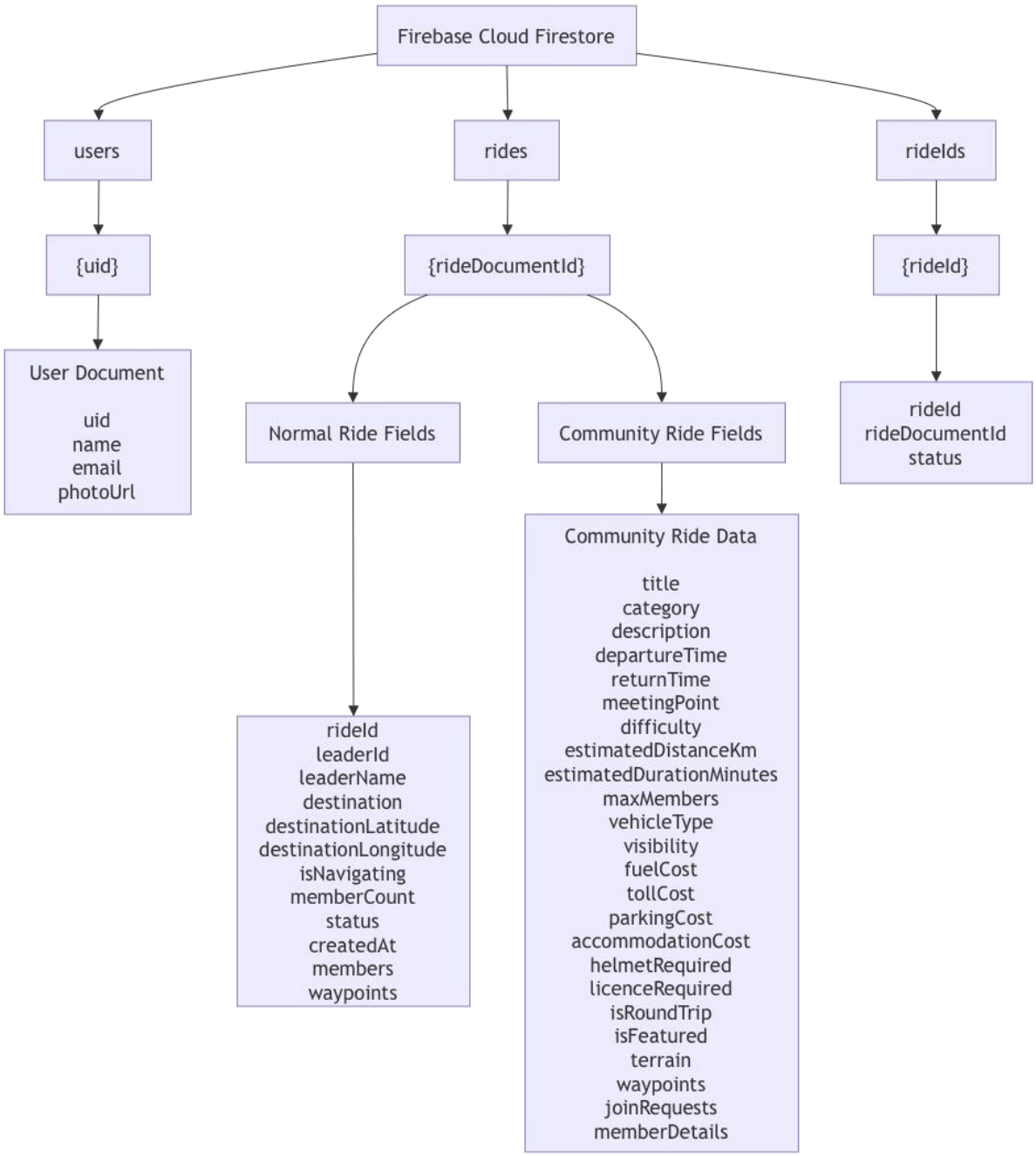


Figure 8 - Database Schema

CHAPTER 6

FRONT-END DESIGN

6.1 FRONT-END TECHNOLOGIES

The front end of OpenTrail is developed using Flutter and Dart, which together provide the foundation for building the application's user interface and implementing its interactive functionalities. The front end is responsible for presenting information to users and allowing them to interact with different features of the application, including authentication, ride management, destination search, and map-based navigation.

Flutter provides the framework and user interface components required to construct the application's screens, layouts, buttons, forms, and interactive elements. It follows a widget-based architecture, allowing individual interface components to be created, reused, and organized into complete application screens. This approach helps maintain consistency throughout the application and supports the development of interfaces for both Android and iOS platforms.

Dart is used to implement the application logic and control the behaviour of these interface components. It handles user interactions, manages application data, processes geographical coordinates, and facilitates communication with Firebase services and external APIs. The combination of Flutter and Dart provides the development foundation for OpenTrail's interactive mobile interface.

6.1.1 FLUTTER

Flutter is an open-source UI framework developed by Google for building applications across multiple platforms using a single codebase. It follows a widget-based approach in which interface elements such as buttons, text fields, containers, lists, and navigation components are represented as widgets. These widgets can be combined and customized to create application layouts and interactive user interfaces. Flutter also provides tools for application development, debugging, testing, and deployment across supported platforms [1].

OpenTrail uses Flutter as its primary application development framework. The framework is responsible for constructing the different screens and visual components of the application, including authentication interfaces, ride management screens, community ride sections, and map-based views. Its widget-based architecture allows interface components to be organized into reusable elements, making the application easier to maintain and modify.

Flutter is also used to integrate external packages that provide additional functionality. For example, Flutter Map is used to display interactive maps, while other packages support location tracking and the application's visual design. By using Flutter, OpenTrail maintains a common application structure across Android and iOS platforms while providing an interface adapted to mobile devices.

6.1.2 DART

Dart is a client-optimized programming language developed by Google and used as the primary programming language for Flutter applications. It is an object-oriented language that supports features such as classes, functions, asynchronous programming, and sound null safety. Dart provides the programming structures required to implement application logic, manage data, and handle interactions between different components of a mobile application [2].

In OpenTrail, Dart is used to implement the functional behaviour of the application. It handles user interactions, screen navigation, form submissions, and the processing of information entered through different interfaces. It also manages geographical coordinates received through the device's location services and processes route-related information obtained from external APIs.

Dart's asynchronous programming capabilities are particularly useful when communicating with Firebase services and external APIs. Operations such as retrieving ride information, submitting data, requesting routes, and obtaining weather information may require network communication. Asynchronous operations allow these tasks to be processed without unnecessarily blocking the application's user interface.

Through Dart, the different application components are connected, allowing user actions to trigger the corresponding operations and display the resulting information within the interface.

6.1.3 FLUTTER MAP AND MAPBOX MAP TILES

Flutter Map is an open-source mapping library for Flutter that provides the components required to display interactive maps within mobile applications. It supports different map layers, including tile layers, marker layers, and polyline layers. These layers allow developers to display map imagery, geographical points, and route lines within a single map interface [8].

Mapbox provides mapping services and map tiles that can be used as the visual foundation of an interactive map. Map tiles represent smaller sections of a geographical map that are combined to display larger areas at different zoom levels. Mapbox provides different map styles and tile services that can be integrated into applications [9].

OpenTrail uses Flutter Map to render the interactive map interface, while Mapbox tiles provide the underlying geographical map imagery. The application uses map markers to represent the positions of riders and polylines to visualize generated routes. This allows participants to view their own geographical position, identify the locations of other group members, and understand the route being followed.

The mapping interface is one of the primary components of OpenTrail because real-time location awareness and route visualization are essential to group ride coordination. The application also incorporates a map tile caching mechanism that stores previously accessed map content locally. This provides limited access to cached map tiles when internet connectivity is unavailable, although complete offline map availability is not guaranteed.

6.1.4 LIQUID GLASS WIDGETS

Liquid Glass Widgets is a Flutter package available through pub.dev that provides reusable interface components for creating liquid-glass-style designs. It includes visual elements that use transparency, background effects, and glass-like surfaces to create a distinctive interface appearance. The package provides components that can be integrated into Flutter applications to implement this visual design approach [3].

OpenTrail uses the liquid_glass_widgets package to implement its selected interface design across the application. The package provides reusable visual components that help maintain a consistent appearance for interface elements such as cards, buttons, and other interactive components.

The liquid-glass design is used to present application information while maintaining a visually consistent interface. It supports the overall appearance of OpenTrail by combining translucent surfaces and visual effects with the application's dark-themed interface. Using reusable widgets also reduces the need to independently construct similar visual components throughout the application.

6.1.5 OPENROUTESERVICE DIRECTIONS API

OpenRouteService is a location-based service that provides routing and direction-related functionality through its APIs. The Directions API calculates routes between geographical locations based on the selected transport profile. It accepts location coordinates as input and returns route information, which may include route geometry, distance, estimated duration, and navigation instructions. The service also supports GeoJSON responses for representing geographical route data [10].

OpenTrail integrates the OpenRouteService Directions API to generate routes between selected locations. When a user selects a destination, the application uses the available geographical coordinates

to request a route from the service. The returned response contains route information that is processed by the application.

The route geometry obtained from the API is converted into geographical coordinates and displayed on the map using Flutter Map's polyline layer. The application also uses the returned distance and duration information to present route-related details to users. During navigation, the current GPS position is used to calculate route progress and update information such as remaining distance and estimated travel time.

The integration of OpenRouteService provides the routing functionality required by OpenTrail without requiring the application to implement its own route calculation engine.

6.1.6 GEOLOCATOR

Geolocator is a Flutter plugin that provides access to the location services available on Android and iOS devices. It allows applications to retrieve the current geographical position, obtain the last known location, and subscribe to continuous location updates. The package also provides utilities for checking location service availability, calculating distances between geographical positions, and determining bearings [11].

OpenTrail uses Geolocator to access the device's GPS functionality and obtain the current geographical position of the rider. The location information is represented using latitude and longitude coordinates, which are used by the application for map positioning and location sharing.

During an active ride, Geolocator provides location updates that allow the application to monitor the movement of participants. These updates are used to maintain the rider's position on the map and support the sharing of location information with other members of the ride. The current geographical position is also used in route-progress calculations to determine the rider's movement along the planned route.

Since location tracking requires access to device location services, the application must obtain the appropriate permissions before retrieving location information. The availability and accuracy of GPS updates may depend on device settings, environmental conditions, and location service availability.

6.1.7 OPENWEATHER API

OpenWeather provides weather-related services through APIs that allow applications to retrieve current weather conditions and forecast information. Its Current Weather Data API accepts geographical coordinates or supported location parameters and returns weather information in formats

such as JSON. The response may contain details such as temperature, atmospheric conditions, humidity, wind speed, and weather descriptions [13].

OpenTrail integrates the OpenWeather API to retrieve weather information for display within the application. The application uses geographical location information to request weather data from the service. The returned response is processed to obtain the relevant weather details and present them through the user interface.

The integration provides riders with weather information related to their current or selected location. This allows users to view available weather conditions while using the application to organize or coordinate their journeys. The API operates through HTTP requests and requires a valid API key to access the weather service.

6.1.8 SUPPORTING PACKAGES: HTTP, ENVIRONMENT CONFIGURATION, AND COORDINATES

In addition to the main frameworks and services, OpenTrail uses supporting Dart and Flutter packages to handle network communication, application configuration, and geographical data processing. These packages provide common utilities required by different application components.

HTTP Package

The http package provides a platform-independent API for performing HTTP requests from Dart applications. It supports communication with web services using methods such as GET and POST and allows applications to receive and process server responses [14].

OpenTrail uses the HTTP package to communicate with external services such as OpenRouteService and OpenWeather. It sends requests containing the required parameters and processes the responses returned by these services. The retrieved information is then used for route generation, weather information, and other API-dependent operations.

Flutter Dotenv

The flutter_dotenv package allows applications to load configuration values from a .env file. It provides a convenient way to access environment variables without directly placing configuration values throughout the application source code [15].

OpenTrail uses flutter_dotenv to load configuration values required by external services, including API keys and other environment-specific settings. This helps separate configuration information from the main application logic and makes it easier to manage service credentials during development.

Latlong2

The latlong2 package provides data structures and utilities for working with geographical coordinates. It includes coordinate representations based on latitude and longitude, along with functions for performing geographical calculations [16].

OpenTrail uses latlong2 to represent geographical positions obtained through GPS services and external routing APIs. These coordinate values are used by the map interface to position rider markers and display route polylines. The package also supports distance calculations between geographical positions, which contribute to the application's route-progress functionality.

Together, these supporting packages assist the main application components by providing network communication, configuration management, and geographical data handling.

6.1.9 WIKIMEDIA API

Wikimedia provides APIs that allow applications to retrieve information and media resources from Wikimedia projects, including Wikipedia and Wikimedia Commons. These services provide access to publicly available content, such as descriptions, image information, and media files, which can be retrieved through API requests.

OpenTrail uses the Wikimedia API to retrieve images associated with geographical locations or places displayed within the application. When users explore location-related information, the retrieved images can be used to provide additional visual context about the selected places. The application processes the API response to obtain the relevant image information and displays the available images within its interface. This integration helps supplement geographical information with visual references, making location-related content more informative for users.

6.1.10 MAPBOX SEARCH BOX API

The Mapbox Search Box API provides location search and autocomplete functionality, allowing applications to retrieve suggestions for geographical places based on user-entered queries. It helps users identify locations and obtain relevant geographical information through API requests.

OpenTrail uses the Mapbox Search Box API to support location and place searches within the application. It helps users find relevant geographical locations by entering search queries and selecting suitable results. The selected location information is then used by the application for destination selection and related map operations.

6.2 FORMS AND INPUT VALIDATION

Forms and input fields are important components of OpenTrail because they allow users to provide the information required for different application operations. The application collects information related to user activities, ride participation, community ride scheduling, and destination selection through its interface. The information entered by users is processed by the application and used to perform the corresponding operations.

OpenTrail provides different input interfaces depending on the functionality being accessed. These include fields for entering ride invitation codes, forms for creating community rides, and destination search fields for selecting geographical locations. Each input interface is designed to collect the information required for its specific operation.

Community Ride Form

The Community Ride form allows users to provide the necessary details when scheduling an upcoming group ride. It collects information such as the ride title, description, departure date and time, meeting point, difficulty level, and other ride-related details supported by the application.

The information entered through this form is used to create a community ride that can be viewed by other users. Interested participants can explore the scheduled ride details and join the ride before its planned departure. The form therefore supports the community ride scheduling functionality by allowing organizers to publish information about upcoming journeys.

Ride Invitation Code Input

The invitation code input is used when a participant wants to join an existing ride session. The user enters the invitation code associated with the ride, allowing the application to identify the corresponding ride information.

The application checks the entered code against the expected format and verifies whether it corresponds to an existing ride. This helps ensure that the joining operation is associated with a valid ride session. The invitation code mechanism provides a way for participants to access and join organized rides.

Destination Search Input

The destination search interface allows users to search for geographical locations and select their intended destinations. Users enter a place name or location-related query, and the application retrieves matching suggestions through the integrated geocoding service.

After selecting a valid destination, the application obtains the corresponding geographical coordinates. These coordinates are used as input for route generation through the OpenRouteService Directions API. This process connects destination selection with the application's mapping and navigation functionality.

6.2.1 INPUT VALIDATION

Input validation is the process of checking information entered by users before it is accepted or used by the application. It helps ensure that the supplied information satisfies the requirements of a particular operation. In OpenTrail, validation is important when users enter invitation codes, submit community ride details, and select destinations for route generation.

The application applies validation according to the requirements of each input field and operation.

Required-Field Validation: Community ride forms contain fields that are required to create a scheduled ride. Required-field validation helps identify missing information before the form is submitted. This ensures that the information needed to create the ride is provided by the organizer.

Invitation Code Validation: The invitation code input is checked to ensure that the entered value follows the expected code format. The application also verifies whether the code corresponds to an existing ride before proceeding with the joining operation. This helps prevent the use of incorrectly formatted or unrecognized invitation codes.

Sl.No.	Input Field	Validation Rule	Expected Behaviour
1	Invitation Code	The field must not be empty	Go to previous screen
2	Invitation Code	The code must follow the expected format as "OT-XXXXXX"	Display an error that the code is invalid
3	Invitation Code	The code must correspond to an existing group ride	Display an error if the invitation code does not exist
4	Community Ride Form	All required fields must not be empty	Display an error if the required fields are empty
5	Community Ride Form	If a field is empty	Display an error that the specific field in empty
6	Search Destination	The destination must be selected before starting navigation	Display an error if destination is not set before navigation

Table 6 - Input Validation

Destination Validation: Destination validation ensures that the user selects a valid location before route generation begins. Entering a search query alone is not sufficient to generate a route; the corresponding destination must be selected so that the application can obtain its geographical coordinates.

These validation mechanisms help reduce incomplete submissions and prevent operations from proceeding when the required information is missing or invalid. They also improve the reliability of the application's input-handling process.

6.2.2 ERROR HANDLING AND USER FEEDBACK

Error handling is an important part of application development because it allows the system to respond appropriately when an operation cannot be completed. Errors may occur due to incomplete user input, invalid information, unavailable data, or problems while communicating with external services. OpenTrail uses input validation and application-level checks to identify conditions that may prevent users from completing an operation.

During ride joining, an error may occur when the invitation code is empty, does not follow the expected format, or does not correspond to an existing ride. The application checks the entered information before proceeding with the joining process. Similarly, when creating a community ride, missing required information may prevent the form from being submitted successfully.

Destination selection also requires appropriate input handling. If a user enters a search query but does not select a valid destination, the application cannot proceed with route generation because the required geographical coordinates are unavailable.

User feedback is important in these situations because it helps users understand why an operation could not be completed and what information needs to be corrected. Appropriate feedback makes the interaction process clearer and reduces confusion when using the application's different features.

In addition to input-related conditions, OpenTrail depends on network connectivity and external services for operations such as retrieving routes, weather information, and cloud-based ride data. Failures in these services may affect the completion of the corresponding operations. Handling such conditions appropriately helps maintain a more predictable user experience.

6.3 SCREENSHOTS OF KEY SCREENS

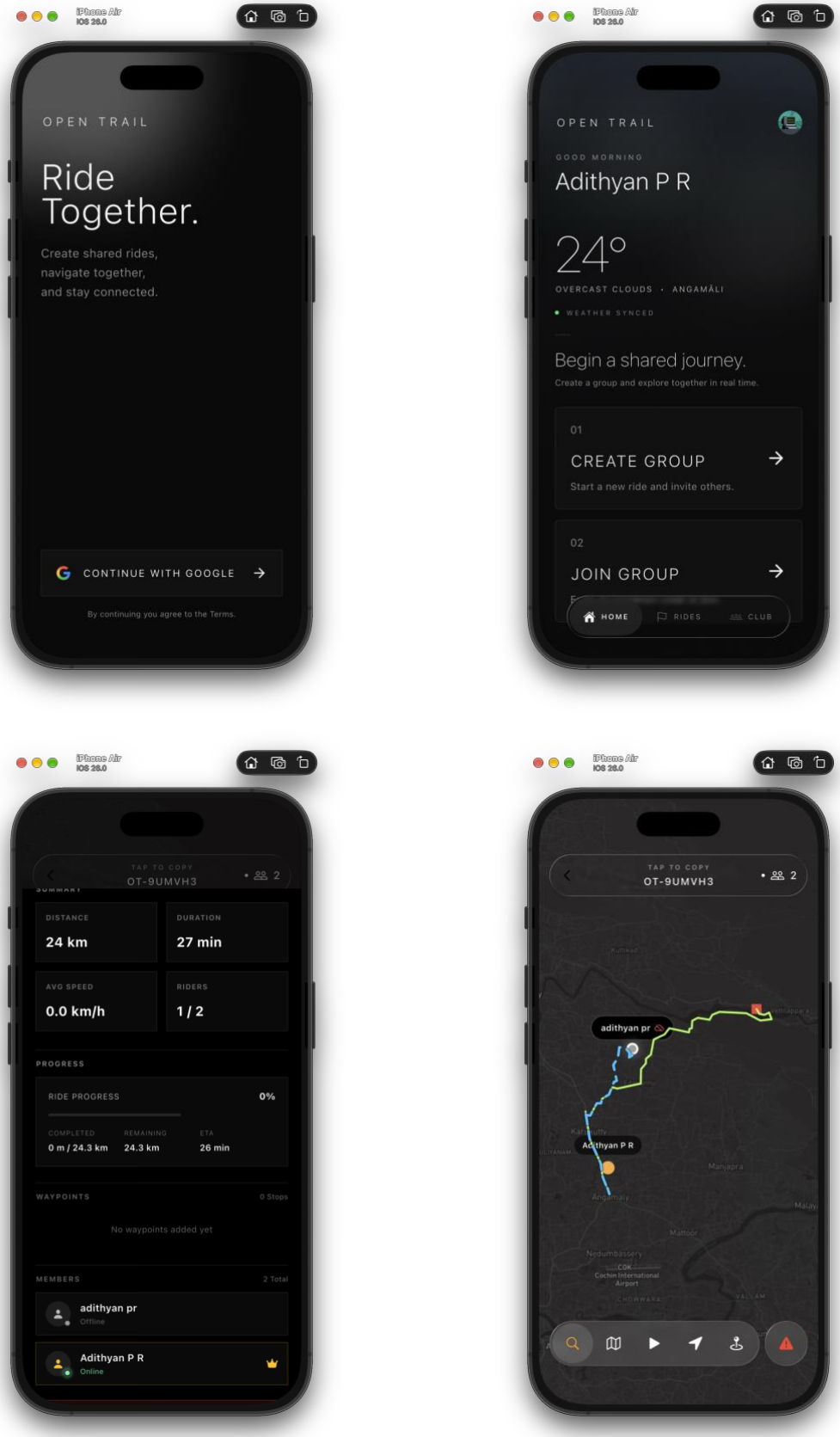


Figure 9 - Open Trail Screen Shots

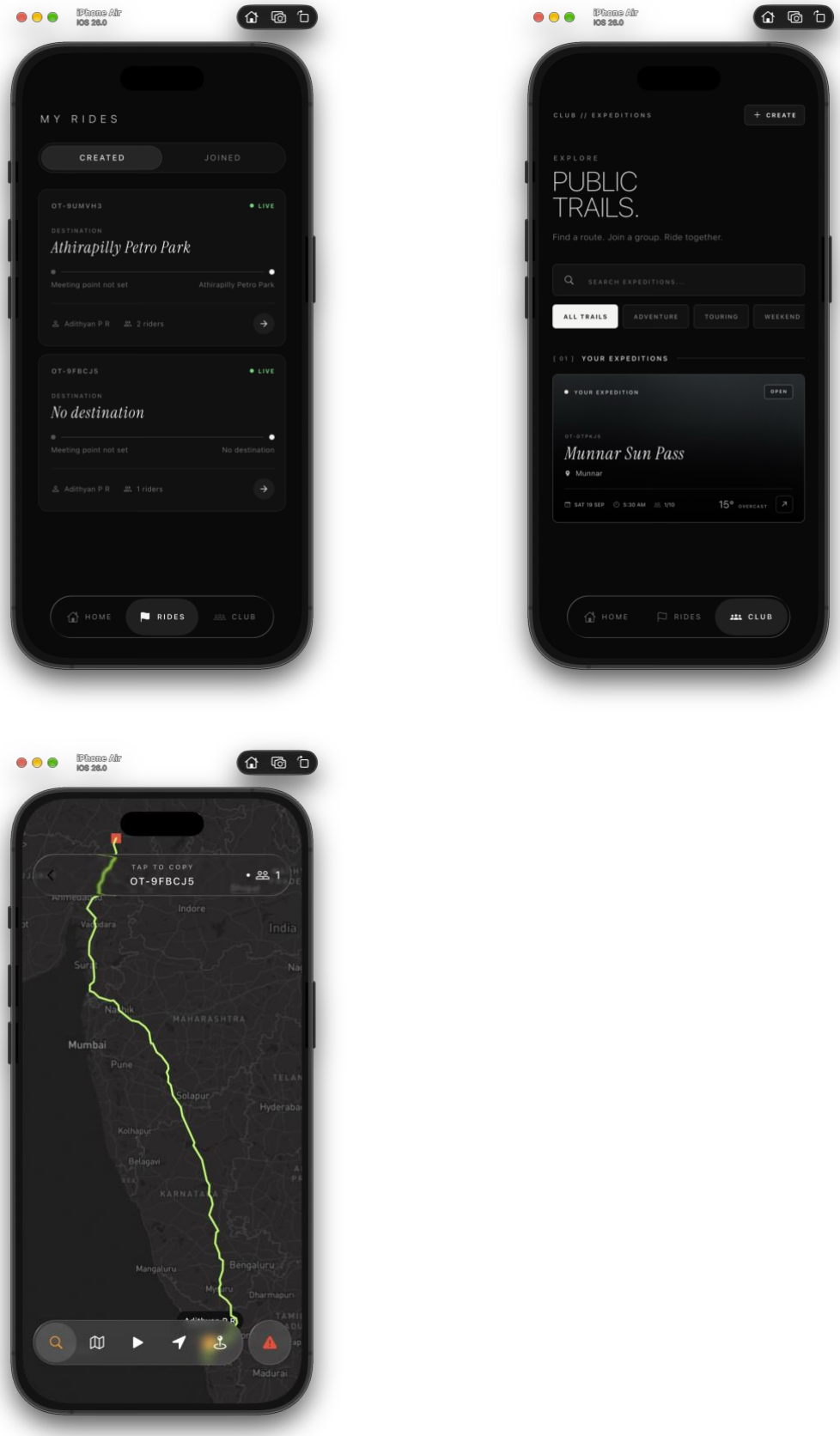


Figure 10 - Open Trail Screen Shots

CHAPTER 6

BACK-END DESIGN

7.1 DATABASE MANAGEMENT SYSTEM

OpenTrail uses Firebase as its primary backend platform, with Cloud Firestore and Firebase Realtime Database serving different data management requirements. Cloud Firestore is a NoSQL cloud database that stores application information in collections and documents. Unlike traditional relational databases, it does not organize information using fixed tables and rows. Instead, data is structured into collections containing individual documents, each of which can store multiple fields and associated values.

Cloud Firestore is used in OpenTrail to manage relatively persistent application information, including user profiles, ride session details, ride identifiers, and community ride information. Its document-based structure allows different types of application data to be organized into separate collections. The Flutter application communicates with Firestore through the Firebase SDK to create, retrieve, update, and manage the information required by different application features.

Firebase Realtime Database is used separately to support the application's real-time location-sharing functionality. It stores data as a JSON tree and synchronizes changes between connected clients. This makes it suitable for handling frequently changing GPS location information during active ride sessions. By separating general application data from frequently updated location information, OpenTrail uses the two Firebase database services according to their respective purposes.

The Cloud Firestore database consists of four main collections: users, rides, rideIds, and community_rides. Each collection is responsible for maintaining a specific category of application information.

1. Users Collection

The users collection stores information associated with registered users of the application. Each user is represented by an individual document identified by the user's unique Firebase Authentication UID. The document contains fields such as uid, name, email, and photoUrl.

The UID provides a unique reference for identifying users within the application, while the name and email fields maintain their basic profile information. The photoUrl field stores the reference to the user's profile image. This collection allows the application to retrieve and manage user information when displaying profiles and associating participants with ride sessions.

2. Rides Collection

The rides collection stores information related to active group ride sessions. Each ride is represented by a document containing the information required to identify, manage, and coordinate the session.

The collection includes fields such as the ride ID, leader ID, leader name, destination, destination coordinates, navigation status, member count, ride status, and creation timestamp. It also maintains member information and waypoints associated with the ride.

These fields allow the application to identify the ride leader, retrieve the destination, determine the current status of the session, and maintain information about participating members. The destination coordinates support map positioning and route-related operations, while the waypoints represent additional geographical points associated with the journey.

The collection plays an important role in ride creation, joining, and management by maintaining the information required throughout an active group ride.

3. RideIds Collection

The rideIds collection maintains ride identification information and its associated status. It includes fields such as the ride document ID and ride status, allowing the application to associate a ride identifier with the corresponding ride information.

This collection supports the identification and retrieval of ride sessions during operations such as joining an existing ride. It provides an organized way to maintain references to ride documents and their current status within the application.

4. Community_Rides Collection

The community_rides collection stores information about upcoming group rides created through the Community Ride feature. Unlike regular active ride sessions, community rides are intended to be scheduled in advance so that other interested users can discover and join them.

The collection contains fields such as the ride title, category, description, departure time, return time, meeting point, difficulty level, estimated distance, estimated duration, maximum number of members, vehicle type, visibility, and cost estimates.

Additional fields include ride requirements, round-trip status, terrain information, waypoints, join requests, and member details. These fields provide information about the planned journey and its participation requirements.

The application uses this information to display scheduled community rides and allow interested users to view ride details and participate by joining the available rides. The collection therefore supports the community-based planning and participation functionality of OpenTrail.

The separation of information into these four collections provides a structured approach to managing different categories of application data. User profiles, active ride sessions, ride identification details, and scheduled community rides are maintained independently according to their functional requirements. Firebase Realtime Database complements this structure by handling live GPS location updates separately from the main Firestore collections.

Figure 6.1 illustrates the database schema and the principal fields associated with the collections used in OpenTrail.

7.2 SERVER ENVIRONMENT

OpenTrail uses Firebase's cloud infrastructure to provide its backend functionality rather than maintaining a dedicated application server using technologies such as Node.js, Django, or other server-side frameworks. Firebase provides managed backend services that can be directly integrated into Flutter applications through its official SDKs.

The Flutter application communicates with Firebase services to perform operations such as user authentication, storing and retrieving ride information, managing community rides, and sharing live location updates. Firebase Authentication handles the user authentication process, while Cloud Firestore and Firebase Realtime Database manage the application's stored information and real-time data synchronization.

This architecture reduces the need to develop and maintain a separate backend server. The application communicates directly with the integrated Firebase services, allowing data operations to be performed through the corresponding SDK methods. External services such as OpenRouteService and OpenWeather are accessed separately through HTTP requests.

Firebase's cloud-based infrastructure also provides managed hosting and data storage capabilities, reducing the infrastructure management responsibilities associated with operating a dedicated server. However, the application's backend operations depend on internet connectivity, Firebase service availability, and the appropriate configuration of authentication and database access permissions.

The server environment of OpenTrail therefore consists primarily of Firebase-managed backend services integrated with the Flutter mobile application. This approach provides the necessary

infrastructure for authentication, application data management, and real-time location sharing without requiring a separately maintained application server.

7.3 FRAMEWORKS AND BACK-END SERVICES

OpenTrail uses Firebase services to implement the backend functionality required by the application. These services are integrated into the Flutter project using their respective packages and SDKs. Each service performs a specific role in managing user access, application data, and communication between participants.

The primary Firebase services used in OpenTrail are Firebase Authentication, Cloud Firestore, and Firebase Realtime Database. Together, these services support the application's authentication process, persistent data management, and live location-sharing functionality.

7.3.1 FIREBASE AUTHENTICATION AND GOOGLE SIGN-IN

Firebase Authentication is a backend authentication service that provides functionality for verifying user identities and managing authentication sessions. It supports different authentication methods, including email and password authentication and third-party identity providers such as Google. Firebase also provides SDKs that allow authentication functionality to be integrated directly into Flutter applications [4].

OpenTrail uses Firebase Authentication to manage user sign-in and provide authenticated access to the application's features. Google Sign-In is integrated as an authentication method, allowing users to authenticate using their Google accounts.

During the sign-in process, the application initiates the Google authentication flow and obtains the required authentication credentials. These credentials are then used with Firebase Authentication to authenticate the user. Once authentication is successful, Firebase provides a unique user identifier (UID) that can be used to associate the authenticated user with their corresponding profile information in Cloud Firestore.

The authenticated user information is used throughout the application to identify participants and associate them with ride-related activities. This integration eliminates the need for OpenTrail to implement and maintain a separate user authentication mechanism.

7.3.2 CLOUD FIRESTORE

Cloud Firestore is a flexible, scalable NoSQL cloud database provided by Firebase. It stores information using collections and documents, where each document contains fields representing

different data values. Firestore provides functionality for querying, retrieving, adding, and updating application data through its client SDKs [6].

OpenTrail uses Cloud Firestore as the primary database for maintaining application information that needs to be stored and retrieved during different operations. The database is organized into four main collections: users, rides, rideIds, and community_rides.

The users collection maintains registered user profile information, while the rides collection stores active ride session details, including the leader, destination, participants, and ride status. The rideIds collection maintains ride identification references, and the community_rides collection stores information about upcoming scheduled rides.

The Flutter application interacts with Firestore through its integrated Firebase SDK. When users perform operations such as creating a ride, joining a session, or scheduling a community ride, the corresponding information is stored or retrieved from the appropriate collection.

Firestore therefore acts as the primary storage system for the structured application data required by OpenTrail. Its collection-based organization allows information associated with different application features to be maintained separately while remaining accessible through the integrated Firebase services.

7.3.3 FIREBASE REALTIME DATABASE

Firebase Realtime Database is a cloud-hosted NoSQL database that stores information in a JSON-based structure. It synchronizes data changes between connected clients in real time, allowing applications to receive updated information without repeatedly requesting the entire dataset. Firebase provides SDKs that allow mobile applications to read and write data and listen for changes in the database [7].

OpenTrail uses Firebase Realtime Database specifically for sharing live GPS location updates among participants during active ride sessions. The application obtains geographical coordinates through the device's location services and publishes the location information to the Realtime Database.

When a participant's location changes, the corresponding update is made available through the database. Other connected participants can listen for these changes and retrieve the updated location information. The application then uses the received coordinates to update the corresponding rider markers on the shared map.

This functionality is particularly important for group ride coordination because it allows participants to monitor the geographical positions of other riders during an active journey. The ride leader can also use the shared map to maintain awareness of the group's movement.

By using Firebase Realtime Database for frequently changing GPS information and Cloud Firestore for general application data, OpenTrail separates its real-time location-sharing requirements from its main data storage operations. This division supports the different data management needs of the application.

7.3.4 DATA FLOW BETWEEN FIREBASE SERVICES

OpenTrail uses different Firebase services to manage authentication, application data, and real-time location updates. These services work together to support the application's major operations. When a user signs in, Firebase Authentication verifies the user's identity and provides a unique user identifier. This identifier is used to associate the user with the corresponding profile information stored in Cloud Firestore.

When a user creates or joins a ride, the application communicates with Cloud Firestore to store or retrieve the relevant ride information. Similarly, scheduled community rides are maintained in the community_rides collection, allowing other users to retrieve and join available rides.

During an active ride, the application obtains the participant's geographical coordinates through GPS and publishes location updates to Firebase Realtime Database. Other participants receive these updates through database listeners, allowing their map interfaces to display updated rider positions. This separation of responsibilities allows OpenTrail to manage general application information and frequently changing location data through their respective Firebase services.

7.3.5 SUMMARY OF BACK-END SERVICES

The backend architecture of OpenTrail combines Firebase Authentication, Cloud Firestore, and Firebase Realtime Database to support the application's major operations. Each service performs a separate responsibility while contributing to the overall functionality of the system. Authentication manages user identity, Firestore maintains application information, and Realtime Database supports live location sharing. This arrangement provides the backend infrastructure required for ride creation, community ride management, participant coordination, and real-time tracking.

CHAPTER 8

SYSTEM IMPLEMENTATION

System implementation is the phase in which the proposed system is developed and configured using the selected technologies and development tools. It involves converting the system design into a functional application by implementing the required features, integrating external services, managing source code, and testing the application during development.

OpenTrail was implemented as a cross-platform mobile application using Flutter and Dart. The development process involved implementing user authentication, ride management, real-time GPS tracking, map integration, route navigation, community ride scheduling, and other supporting functionalities. Firebase services were integrated to manage application data and location updates, while external APIs were used to provide routing, weather, and location-related information.

This chapter describes the development environment, version control system, Git commands, and development history associated with the implementation of OpenTrail.

8.1 INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)

OpenTrail was developed using Visual Studio Code (VS Code) as the primary Integrated Development Environment. It provides a convenient workspace for writing, editing, debugging, and managing Flutter and Dart source code. The IDE supports Flutter development through dedicated extensions and provides tools for executing, debugging, and testing the mobile application.

Visual Studio Code includes several development features such as syntax highlighting, code completion, error detection, and debugging support. These features help developers identify coding errors, understand the application structure, and improve development efficiency. The Flutter and Dart extensions provide additional functionality, including code analysis, widget assistance, debugging tools, and access to commonly used Flutter development commands.

During the development of OpenTrail, VS Code was used to organize project files, modify application components, and implement different modules. These included user authentication, ride creation and management, interactive map integration, GPS-based location tracking, route navigation, and Firebase connectivity. The editor also supported the development and modification of reusable Flutter widgets, allowing different interface components to be maintained and updated efficiently.

The integrated terminal available in Visual Studio Code was used to execute Flutter commands and manage the development workflow. Commands such as flutter pub get, flutter run, and other Flutter

CLI operations were used to retrieve project dependencies, execute the application, and identify build-related issues. The terminal also provided access to Git commands for managing source code changes.

Another important feature of the IDE is its support for an iterative development process. Developers can modify the source code, run the application, observe the output, and correct implementation issues repeatedly. Flutter's hot reload functionality also helps developers view many interface changes without restarting the entire application. This was useful when refining the map interface, adjusting layouts, and improving the overall user experience.

Overall, Visual Studio Code provided the primary development environment for building, debugging, testing, and maintaining the OpenTrail application. Its integration with Flutter, Dart, and Git helped maintain an organized and efficient development workflow.

8.2 VERSION CONTROL SYSTEM

Git was used as the version control system for managing the OpenTrail source code. It is a distributed version control system that records changes made to project files over time. It allows developers to maintain different versions of the source code, review previous modifications, and restore earlier versions when required. Git also provides a structured approach to managing development changes and maintaining the project throughout its development lifecycle. [8]

GitHub is a web-based platform used for hosting and managing Git repositories. It provides a centralized online location for storing source code and accessing project history. In OpenTrail, GitHub was used to host the application's source code repository, maintain development records, and provide access to different versions of the project. It also supports collaboration, code sharing, and long-term project maintenance. [9]

Although Git and GitHub are closely related, they serve different purposes. Git manages source code changes and version history locally on the development system, whereas GitHub provides an online platform for hosting Git repositories. Together, they support source code management, backup, and a more organized software development workflow.

8.2.1 MAJOR GIT COMMANDS USED IN THE PROJECT

The following Git commands were used during the development and maintenance of the OpenTrail source code. These commands help initialize the repository, monitor file modifications, record changes, and synchronize the local project with its remote GitHub repository.

Git Command	Purpose
git init	Initializes a new Git repository in the project directory.
git status	Displays the current state of the working directory and staged changes.
git add .	Stages the modified and newly added files for the next commit.
git commit -m "message"	Records the staged changes with a descriptive commit message.
git log --oneline	Displays the commit history in a compact, one-line format.
git push	Uploads local commits to the configured remote repository, such as GitHub.
git pull	Fetches and integrates changes from the remote repository into the local branch.
git clone <repository-url>	Creates a local copy of an existing remote Git repository.

Table 7 - Git Commands

These commands provide the basic operations required for maintaining the project repository. The git add and git commit commands were used to prepare and record development changes, while git push was used to synchronize committed changes with GitHub. The git log --oneline command was useful for reviewing the sequence of modifications made during development.

8.2.2 GITHUB REPOSITORY DETAILS

The OpenTrail source code is maintained in a GitHub repository named **OpenTrail-app**. The repository contains the Flutter application source code and related project files. It provides access to the development history and allows the project to be maintained independently of the local development environment.

The project also has a dedicated website that presents information about the OpenTrail application and its functionality.

Repository Name: OpenTrail-app

Repository URL: https://github.com/Aathi-pr/OpenTrail-app.git

Project Website: OpenTrail – Realtime Convoy Coordination For Group Rides

8.2.3 DEVELOPMENT HISTORY

The development of OpenTrail was carried out in multiple phases. Each phase focused on implementing and improving a particular set of application functionalities. Git commits were used to record important modifications and maintain a chronological history of the development process.

The major development activities are summarized below.

Phase 1 – Project Foundation

- **12 July 2026:** Created the initial project commit and established the basic project structure.
- **14 July 2026:** Integrated Liquid Glass widgets and performed initial application optimizations.
- **14 July 2026:** Optimized the map interface and resolved layout-related issues.

Phase 2 – Navigation and Mapping

- **17 July 2026:** Improved the user interface and implemented turn-by-turn navigation using the OpenRouteService Directions API.
- **18 July 2026:** Added route polylines for riders and improved rider visualization on the map.
- **20 July 2026:** Optimized map performance and implemented rerouting functionality.
- **26 July 2026:** Added ride analytics functionality.
- **27 July 2026:** Refactored and optimized the map_page.dart implementation.
- **28 July 2026:** Enhanced ride analytics and modified map markers.

Phase 3 – Navigation and Safety Features

- **3 August 2026:** Added waypoint functionality to support route planning.
- **4 August 2026:** Introduced the SOS button and resolved related implementation issues.
- **5 August 2026:** Optimized application performance and implemented weather updates.

Phase 4 – Community Features

- **8 August 2026:** Developed the Community page interface and enhanced weather information.
- **23 August 2026:** Implemented ride scheduling functionality within the Community section.

Phase 5 – Project Finalization

- **30 August 2026:** Updated project configuration and excluded Firebase configuration files from version control to improve the handling of sensitive project configuration.
- **31 August 2026:** Updated the project README.md file and supporting documentation.

8.2.4 GIT COMMIT HISTORY

The Git commit history provides a chronological record of the changes made to the OpenTrail source code. Each commit represents a recorded modification to the project, such as adding a new feature, improving an existing component, fixing an issue, or updating project configuration.

The commit history can be reviewed using the git log command in the integrated terminal or through the GitHub repository interface. This helps identify when particular changes were introduced and provides a reference for understanding the application's development progress.

```
cdd3857 (HEAD -> main, origin/main) fixed warnings depricated packages
966b81e Update README.md
820dd91 chore: update project and ignore Firebase config
8a3220e Community Page added with ride sheduling
5b1e46a community page ui created and enhanced weather info
e48bf41 (tag: v1.1.0) optimized performance and implemented weather updates
027f76d sos bugs fixed
ebe1727 added sos button
59ad886 added sos button
7f761f3 (tag: v1.0.0) updated Readme.md
3dd1303 added images for readme.md
d1ecbf4 Added new feature waypoints
7fd78f7 added analytics and modified marker
f65fac6 refactored the map page and optimized the overall map_page.dart
d1cf9ea ride analytics added
23a9878 optimized the map and added rerouting functionality
fa876e6 added polyline to other riders to leader and other improvements
aa158d4 improved ui and added turn by turn navigation using openroute service di
rection API
a59ac9c add liquid_glass widgets and other optimizations
6e16cfe optimized map and fixed layout bugs
38906b4 Initial commit
~
(END)
```

Figure 11 - Git Commit History

CHAPTER 9

TESTING AND VALIDATION

Software testing and validation are important phases in the development of an application. Testing is performed to identify errors, verify the correctness of implemented functionalities, and ensure that the application behaves according to its requirements. Validation helps determine whether the developed system satisfies its intended purpose and provides the expected functionality to users.

OpenTrail was tested at different levels to evaluate the correctness of individual functions, the interaction between integrated components, and the behaviour of the complete application. The testing process focused on major functionalities such as user authentication, ride management, GPS-based location tracking, route generation, SOS indication, and map interaction.

The following sections describe the different testing approaches used to evaluate the functionality and reliability of OpenTrail.

9.1 UNIT TESTING

Unit testing is a software testing technique in which individual functions, methods, and components are tested independently to verify their correctness. It helps identify errors at an early stage of development before the components are integrated into the complete application.

In OpenTrail, unit-level verification focuses on individual operations related to geographical calculations, route information processing, ride data handling, and distance calculations. These operations are important because the application depends on accurate location information and properly processed ride details.

The testing process involves providing valid and invalid inputs to individual functions and observing their outputs. Valid inputs are used to verify whether the functions produce the expected results, while invalid or incomplete inputs help evaluate how the application handles unexpected data.

The following table presents the unit-level test cases considered for OpenTrail.

These test cases help evaluate the reliability of individual operations under different input conditions. Particular attention is given to handling incomplete data, processing route responses, and calculating geographical distances accurately. Identifying errors at this level helps prevent incorrect results from affecting other modules of the application. The outcomes of unit-level verification also provide a foundation for the subsequent integration and system testing phases.

Test Case ID	Function Tested	Test Procedure	Expected Result	Pass / Fail
UT01	Distance Calculation	Provides two coordinates to the distance calculation function	The function calculates the distance correctly	Pass
		Provides invalid coordinates	The function throws an unhandled exception	Fail
UT02	Remaining Distance Calculation	Provide the current location and destination coordinates	The function calculate remaining distance	Pass
		Provide missing location data	The function cannot calculate remaining the distance	Fail
UT03	Route Data Processing	Supply valid route data containing distance, duration and route in route_service.dart	The function extracts the required route information correctly	Pass
		Supply invalid route data containing distance, duration	The function cannot process the route data	Fail
UT04	Ride Data Processing	Provide valid ride details to ride_service.dart	The function process the ride information and expected values	Pass
		If invalid ride details are provided	The function cannot process the data needed	Fail

Table 8 - Unit Testing

Unit testing helps verify the correctness of individual operations used throughout the application. It is particularly important for geographical calculations and route processing because incorrect values can affect navigation information and the display of remaining distance. Testing ride data processing also helps ensure that the information used by the ride management module is handled correctly.

9.2 INTEGRATION TESTING

Integration testing verifies whether different modules and services of an application communicate and exchange information correctly. Individual components may function properly when tested separately, but integration testing is required to identify problems that can occur when they interact with one another.

OpenTrail integrates several components, including the Flutter application, Firebase Authentication, Cloud Firestore, Firebase Realtime Database, GPS functionality, mapping components, and external APIs. These components work together to provide authentication, ride management, real-time location sharing, and route-related functionality.

Integration testing focuses on verifying the communication between these components and ensuring that information is transferred and updated correctly. The following test cases describe the major integration scenarios considered in OpenTrail.

Test Case ID	Components Tested	Test Procedure	Expected Result	Pass / Fail
IT01	Flutter and firebase authentication	Attempt to sign in using a supported authentication method	The authentication is correctly handled and stored in firebase	Pass
		Attempt to sign in when authentication fail	The app fails to authenticate the user and displays an error	Fail
IT02	Ride Management and Firestore	Ride creation details are properly inserted in firebase cloud firestore	The ride information stored and retrieved correctly	Pass
		Attempt to create a ride when the firestore request is rejected	The ride information is not saved successfully	Fail
IT03	Geolocator and Map Interface	Grant the locatio0n permission and observe the current position in the map	The available device location is represented on the map	Pass
		Deny location permission	The current location cannot be displayed	Fail

Test Case ID	Components Tested	Test Procedure	Expected Result	Pass / Fail
IT04	Cloud Firestore and Location Tracking	Update riders location and observe shared map interface	The location update is reflected for all riders in the shared session	Pass
		Attempt to update ride location when the firestore requests fails	The location cannot be sync with other riders	Fail
IT05	SOS State and Map Marker	Activate SOS and inspect the marker displayed for the affected rider	The SOS state is synchronized and displayed correctly.	Pass
		Activate SOS when the SOS state fails to sync	The SOS alert cant be displayed correctly	Fail

Table 9 - Integration Testing

Integration testing helps verify that the different services used by OpenTrail function together as expected. In particular, the interaction between Firebase services and the Flutter application is important for maintaining ride information and sharing live GPS updates. It also helps identify communication issues that may affect the shared map, authentication, and SOS indication.

9.3 SYSTEM TESTING

System testing evaluates the complete application as an integrated system. It verifies whether the implemented functionalities work together correctly and whether the application satisfies its intended requirements.

Unlike unit testing and integration testing, which focus on individual components and their interactions, system testing examines complete user workflows. In OpenTrail, these workflows include signing in, creating a ride, joining an existing ride, viewing participants' locations, generating routes, and managing an active ride session.

Testing includes verifying Google Sign-In to ensure that users can access the application through the supported authentication method. Ride creation and joining functionalities are examined to determine whether users can create new ride sessions and participate in existing ones. The location tracking

feature is evaluated to verify whether participants' geographical positions are displayed on the shared map.

Destination search and route generation using the OpenRouteService Directions API are also considered to verify the application's route-related functionality. The application is further evaluated for its ability to display navigation information, including remaining distance and estimated travel time.

The SOS mechanism is examined to assess whether an emergency indication can be activated and communicated to other participants. Input validation is also considered to determine whether incomplete or invalid information is handled appropriately. Location permissions and network connectivity are important testing conditions because they can affect GPS tracking, Firebase communication, and external API requests.

The following tables present the major system-level test cases considered for OpenTrail.

Test Case ID	Feature	Test Procedure	Expected Result	Pass / Fail
ST01	User Login	Open the application and sign in with google	The user can access the application and successfully authenticated	Pass
		Sign in with google fails	The user cannot be logged in to the application	Fail
ST02	Ride Creation	Create a new ride using the available ride details	The ride is created and the user is redirected to the map page	Pass
		Attempt to create a ride when Firebase fails	The ride cannot be created	Fail
ST03	Ride Joining	Join an existing ride	The user join to the existing ride using the ride ID	Pass
		Attempt to join a ride with invalid ride Id	The app rejects the invalid ride id by showing a message	Fail
ST04	Live Location Tracking	Once joined the users in the shared session can view their and others location on the shared map	Location updates are properly updated in the map	Pass
		Deny location permission and attempt to access live location	The app cannot retrieve location information to the joined session	Fail

Test Case ID	Features	Test Procedure	Expected Result	Pass / Fail
ST05	Ride Analytics	Uses the available information of the ongoing ride and interpret data to show the analytics of both ongoing and end ride analytics	The information is properly generated and retrieved via firebase	Pass
		Attempt to retrieve ride information when firebase fails	The app handles missing data without crashing	Fail
ST06	Offline Caching	Caches the map for offline navigation	Ensures the map can be accessed both network is connected and not connected	Pass
		Attempt to access uncashed map	The application tries to reconnect and cache when network is available	Fail

Table 10 - System Testing

System testing provides an overall evaluation of OpenTrail from the user's perspective. It helps determine whether the major application features operate together and whether the system supports its primary purpose of group ride coordination.

The testing process also highlights the importance of external dependencies such as GPS permissions, internet connectivity, Firebase services, and routing APIs. Since some application functionalities depend on these services, their availability can influence the overall user experience.

Overall, testing and validation help identify implementation issues, verify the expected behaviour of the application's major features, and evaluate whether OpenTrail satisfies its functional requirements.

CHAPTER 10

CONCLUSION

10.1 SUMMARY OF WORK

OpenTrail is a mobile application developed to simplify the coordination of group motorcycle rides and cycling journeys. It is designed for motorcycle riders, cycling groups, friends travelling together, and ride organizers who require a convenient platform to coordinate participants, share location information, and follow planned routes. During group journeys, participants may become separated, lose track of other riders, or experience difficulties following the planned route. OpenTrail aims to address these challenges by integrating essential group ride coordination functionalities into a single mobile application.

The application was developed using Flutter and Dart to provide a cross-platform mobile experience. Firebase Authentication with Google Sign-In is used to manage user authentication, while Cloud Firestore stores user profiles, ride details, and community ride information. Firebase Realtime Database supports real-time GPS location updates among participants. Flutter Map provides the interactive map interface with Mapbox map tiles, while OpenRouteService is integrated to generate route information. Geolocator supports device location tracking, and destination search functionality helps users identify their intended destinations. Additional services, including OpenWeather, provide weather-related information within the application.

OpenTrail allows users to create ride sessions, invite or join other participants, and view the locations of group members through a shared map interface. Users can search for destinations, access route information, and coordinate their journeys with other participants. Ride organizers can manage ride sessions and monitor participating riders during an active journey. The application also includes an SOS mechanism that allows a ride leader to indicate an emergency situation to other participants. These functionalities help improve group awareness and make ride coordination more convenient.

The application supports a workflow in which users sign in using Google Sign-In, create a new ride or join an existing ride, and share their location after granting the required location permissions. Participants can access the shared map to view available locations of other riders, search for destinations, and obtain route information. The application also provides navigation-related details, including remaining distance and estimated travel time. Ride organizers can use the available ride management features to coordinate participants, while the SOS mechanism provides a way to communicate an emergency indication within the group. The availability of location and ride

information depends on the required permissions, network connectivity, and supporting application services.

The development of OpenTrail provided practical experience in cross-platform mobile application development, cloud database integration, GPS-based location tracking, interactive mapping, and external API integration. It also involved understanding how multiple services can be integrated into a single application to support different operational requirements. The implementation process provided opportunities to work with Flutter widgets, Firebase services, location-based functionalities, route processing, and version control using Git and GitHub.

Overall, OpenTrail demonstrates how mobile application technologies and location-based services can be applied to improve group travel coordination. By combining ride management, shared location tracking, destination search, route information, weather updates, and emergency indication, the application provides a unified platform for motorcycle and cycling communities. The project establishes a foundation that can be further developed through additional navigation, communication, safety, and ride management capabilities.

10.2 FUTURE SCOPE

Although OpenTrail provides essential functionalities for group ride coordination, several improvements and additional features can be considered in future versions to enhance its usability, reliability, and overall performance. These improvements can extend the existing capabilities of the application and provide a more comprehensive experience for motorcycle and cycling communities.

1. Enhanced Navigation

The navigation system could be improved by providing more detailed route guidance, accurate route progress calculation, and additional information about upcoming turns and road conditions. Further improvements to route recalculation could help participants adapt more effectively when they move away from the planned route.

2. Improved SOS Functionality

The existing emergency indication mechanism could be enhanced by introducing more reliable synchronization and immediate notifications to other ride participants. Future versions could also support sharing additional emergency details, such as the affected rider’s location and the nature of the emergency, to improve awareness and coordination during critical situations.

3. Improved Communication

An in-app communication system could be introduced to allow riders to exchange important information without depending entirely on separate messaging applications. Features such as group messaging and voice communication could improve coordination between ride leaders and participants, particularly during long journeys.

4. Advanced Group Coordination

Additional group monitoring features could be developed to identify participants who fall significantly behind the group or remain stationary for an extended period. The application could notify the ride leader when such situations occur, helping the group respond more quickly and maintain better coordination throughout the journey.

5. Performance and Security Improvements

Further improvements could focus on optimizing database operations, reducing unnecessary network requests, and improving application responsiveness. Additional security measures could also be introduced to strengthen access control and protect user and ride information. Performance optimization would help provide a more consistent experience, particularly when multiple participants are sharing location updates simultaneously.

6. Enhanced Offline Support

Future versions could introduce more comprehensive offline map support by allowing users to download selected geographical areas before beginning a journey. This could improve map accessibility in locations with limited network connectivity. Additional improvements could also focus on retaining essential route information and synchronizing relevant data when connectivity becomes available again.

These enhancements could extend the capabilities of OpenTrail and make it a more comprehensive platform for group ride planning, navigation, communication, and safety. Continued development could further improve its usefulness for motorcycle riders, cycling groups, and organized riding communities.

CHAPTER 11

REFERENCES

11.1 TECHNICAL DOCUMENTATION

The following technical documentation and official resources were referred to during the design, development, and implementation of the OpenTrail application. These references provide information about the frameworks, programming languages, cloud services, mapping technologies, and external APIs used in the project.

[1] Flutter, "Flutter Documentation." Available: https://docs.flutter.dev/

[2] Dart, "Dart Documentation." Available: https://dart.dev/overview

[3] Firebase, "Firebase Authentication for Flutter." Available: https://firebase.google.com/docs/auth/flutter/start

[4] Firebase, "Cloud Firestore Documentation." Available: https://firebase.google.com/docs/firestore

[5] Firebase, "Firebase Realtime Database Documentation." Available: https://firebase.google.com/docs/database

[6] Flutter Map, "Flutter Map Documentation." Available: https://docs.fleaflet.dev/

[7] Mapbox, "Maps API Documentation." Available: https://docs.mapbox.com/api/maps/

[8] openrouteservice, "Directions API Documentation." Available: https://openrouteservice.org/dev/

[9] Geolocator, "Geolocator Flutter Package." Available: https://pub.dev/packages/geolocator

[10] Geocode Earth, "Autocomplete API Documentation." Available: https://geocode.earth/docs/forward/autocomplete

[11] OpenWeather, "Current Weather Data API." Available: https://openweathermap.org/api/current

[12] liquid_glass_widgets, "Flutter Package Documentation." Available: https://pub.dev/packages/liquid_glass_widgets

DIGITAL ACCESS

The digital resources associated with the OpenTrail project can be accessed by scanning the QR code provided below. It provides convenient access to the project's online documentation and related digital resources.

Open Trail – Digital Documentation

https://adithyanpr.site/work/open-trail/index.html

(DISPLAYBARCODE " https://adithyanpr.site/work/open-trail/index.html

" QR \q 3)